

PC FORGE

Ghidul final de invatat pentru prezentare

Pachetul selectat: 8,275 puncte din Etapele 5, 6 si 7

Obiectiv: Sa poti arata functionalitatea, sa urmaresti datele pana in cod si sa raspunzi natural la intrebarea «de ce ai implementat asa?». Ghidul nu cere memorarea codului caracter cu caracter.

Popa Bogdan | PC Forge | material de studiu si demonstratie

Cum inveti eficient din acest ghid

Pachetul valoreaza aproximativ 8,275 puncte. Din el ai nevoie de aproximativ 4,98 puncte pentru nota 10. Invata mai intai demonstratia si ideea fiecarei functii, apoi citeste fragmentele de cod. Fiecare capitol iti spune exact ce fila deschizi, ce apesi si ce explici.

Formula universală de raspuns: Cerinta cere X. Datele pornesc din Y. Functia Z le transforma astfel. Rezultatul este W, pe care il demonstrez in browser. Am ales aceasta solutie deoarece respecta cerinta si separa responsabilitatile.

Pregatirea calculatorului

Porneste Docker Desktop si asteapta ca PostgreSQL PC Forge sa fie disponibil.

Deschide proiectul C:\Users\bogdi\Desktop\PC-Forge in VS Code si porneste serverul cu npm start.

Deschide <http://localhost:8080>, <http://localhost:8080/produse> si o fereastră privata pentru bannerul cookie.

In VS Code foloseste cautarea dupa numele functiei. Nu derula la intamplare prin index.js.

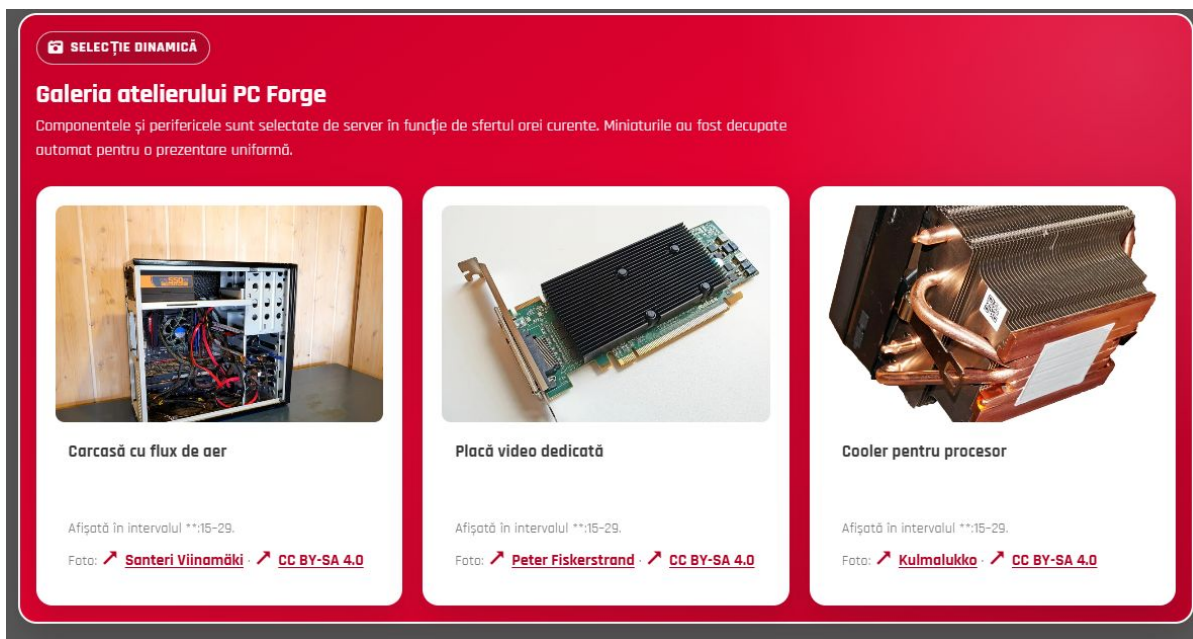
Nu afisa fisierul .env, parole, date SMTP sau hashuri reale in fata clasei.

Ordinea prezentarii

Incepi cu rezultatul vizibil. Dupa ce profesoara vede ca functioneaza, treci in VS Code si arati numai functia relevanta. Revii apoi in browser. Aceasta ordine dovedeste functionalitatea si evita pierderea timpului in cod.

ETAPA 5 - pachet selectat: 1,475 puncte

Ideea etapei: Galeria si Bootstrap-ul arata rezultatul, iar serverul automatizeaza selectarea imaginilor, compilarea Sass, backupul si validarea resurselor.



Galeria statica PC Forge - rezultatul serverului si al gridului responsive.

5.1 Galeria statica (0,35 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/#galerie-statica> | VS Code: `resurse/json/galerie.json`; `index.js`; `views/fragmente/galerie-statica.ejs`; `resurse/sass/galerie.scss`

Ce cere exact: Imaginile trebuie citite din JSON, filtrate pe server dupa sfertul orei, afisate cu figure si figcaption, dimensionate uniform si organizate responsive in 3, 2 sau 1 coloana.

Demonstratia, in ordinea corecta

Deschide galeria si indica imaginile de aceeasi dimensiune si descrierile lor.

Redimensioneaza fereastra: trei coloane pe ecran mare, doua pe mediu si una pe mic.

Treci cu mouse-ul peste o imagine si arata transformarea, rotirea si modificarea chenarului.

In VS Code arata obiectul JSON, filtrarea pe server si bucla EJS.

Cum functioneaza in proiect

`galerie.json` este sursa de date. Fiecare obiect contine calea imaginii, titlul, descrierea si `sfert_ora`.

`obtainImaginiGalerieStatica` ruleaza pe server. Minutele sunt impartite in intervale de cate 15; valoarea 1 este permanenta, iar celelalte valori aleg sfertul corespunzator.

Serverul transmite template-ului numai imaginile eligibile. EJS nu primeste toate imaginile pentru a le ascunde ulterior.

`figure` grupeaza semantic imaginea cu `figcaption`, iar CSS Grid controleaza numarul coloanelor.

Algoritmul pe care trebuie sa il intelegi

Citeste minutul curent.

Calculeaza sfertul orei prin `Math.floor(minute / 15) + 1`.

Pastreaza imaginile permanente sau cele care corespund sfertului.

EJS parcurge vectorul cu `forEach` si produce markupul.

Media queries schimba numarul coloanelor; tranzitia anima proprietatile de hover.

Bucata-cheie: functia care face filtrarea pe server:

```
479 | function obtineImaginiGalerieStatica() {
480 |   const sfertOraCurent = Math.floor(new Date().getMinutes() / 15) + 1;
481 |
482 |   return obGlobal.obGalerie.imagini
483 |     .filter(function (imagine) {
484 |       return imagine.sfert_ora === sfertOraCurent && imagine.cale_thumbnail;
485 |     })
486 |     .slice(0, 10);
487 | }
488 |
489 | /**
490 |  * Alege aleator un grup consecutiv de imagini pentru galeria animata.
491 |  * @returns {{imagini: object[], numarImagini: number}} Datele galeriei animate.
```

Intrebari probabile si raspunsuri

De ce nu ascunzi imaginile cu CSS?

Raspuns: Cerinta spune ca serverul trebuie sa trimita numai imaginile relevante. CSS ar primi oricum toate datele.

De ce figure si nu doar div?

Raspuns: figure si figcaption exprima semantic faptul ca legenda apartine imaginii.

Ce inseamna responsive?

Raspuns: Componentele se reorganizeaza dupa spatiul disponibil; nu este doar micșorarea întregii pagini.

5.2 Compilarea automata SCSS (0,25 p)

Ce trebuie sa deschizi: Browser: Terminalul VS Code si orice pagina a site-ului | VS Code: index.js; resurse/sass; resurse/css; backup/resurse/css

Ce cere exact: Serverul defineste folderele SCSS/CSS/backup, compileaza toate sursele la pornire, salveaza CSS-ul anterior si recompila automat fisierul modificat.

Demonstratia, in ordinea corecta

Arata mesajele de compilare din terminal la pornire.

Modifica o valoare nepericuloasa dintr-un SCSS si salveaza.

Arata aparitia mesajului de recompilare si apoi schimbarea in browser.

Deschide backup/resurse/css si arata copia precedenta.

Cum functioneaza in proiect

SCSS este sursa pentru dezvoltare; browserul citeste CSS-ul rezultat.

compileazaScss rezolva caile absolute sau relative, creeaza folderul destinatie, face backup si apeleaza sass.compile.

compileazaToateScss parcurge sursele la pornire. Fisierul partial care incepe cu underscore sunt importate de alte fisier si nu trebuie compilate separat.

urmarestescss foloseste fs.watch. Debounce-ul evita mai multe compilari pentru aceeasi salvare.

Algoritmul pe care trebuie sa il intelegi

Determina calea sursa si destinatia.

Daca exista CSS vechi, il copiaza in backup.

Compileaza SCSS cu pachetul sass.

Scrie CSS-ul rezultat.

Watcherul observa modificarile si reapele functia dupa o intarziere scurta.

Bucata-cheie: inceputul functiei compileazaScss si fluxul backup-compilare:

```

221 | function compileazaScss(caleScss, caleCss) {
222 |   let caleScssAbs;
223 |   let caleCssAbs;
224 |
225 |   // Daca fisierul SCSS are cale absoluta, o folosim direct.
226 |   // Daca are cale relativa, o consideram relativa la folderScss.
227 |   if (path.isAbsolute(caleScss)) {
228 |     caleScssAbs = caleScss;
229 |   } else {
230 |     caleScssAbs = path.join(obGlobal.folderScss, caleScss);
231 |   }
232 |
233 |   // Daca s-a transmis calea CSS, o folosim.
234 |   // Altfel, generam automat numele CSS pornind de la numele SCSS.
235 |   if (caleCss) {
236 |     if (path.isAbsolute(caleCss)) {
237 |       caleCssAbs = caleCss;
238 |     } else {
239 |       caleCssAbs = path.join(obGlobal.folderCss, caleCss);
240 |     }
241 |   } else {
242 |     const caleRelativaScss = path.relative(obGlobal.folderScss, caleScssAbs);
243 |     const caleRelativaCss = caleRelativaScss.replace(/\.scss$/i, ".css");
244 |     caleCssAbs = path.join(obGlobal.folderCss, caleRelativaCss);
245 |   }
246 |
247 |   // Cream folderul destinatie pentru CSS, daca nu exista.
248 |   creeazaFolderDacaNuExista(path.dirname(caleCssAbs));
249 |
250 |   // Inainte de suprascriere, salvam CSS-ul vechi in backup.
251 |   if (fs.existsSync(caleCssAbs)) {
252 |     try {
253 |       const caleRelativaCss = path.relative(obGlobal.folderCss, caleCssAbs);
254 |       const infoFisierCss = path.parse(caleRelativaCss);
255 |
256 |       const numeBackup =
257 |         infoFisierCss.name + "_" + Date.now() + infoFisierCss.ext;
258 |
259 |       const caleBackup = path.join(
260 |         obGlobal.folderBackup,
261 |         "resurse",
262 |         "css",
263 |         infoFisierCss.dir,
264 |         numeBackup,
265 |       );
266 |
267 |       creeazaFolderDacaNuExista(path.dirname(caleBackup));
268 |       fs.copyFileSync(caleCssAbs, caleBackup);
269 |
270 |       console.log("Backup CSS creat: " + caleBackup);
271 |     } catch (eroareBackup) {
272 |       console.error("Eroare la crearea backupului pentru: " + caleCssAbs);
273 |       console.error(eroareBackup.message);
274 |     }

```

```

275 | }
276 |
277 | // Compilarea efectiva SCSS -> CSS
278 | try {
279 |   const rezultat = sass.compile(caleScssAbs, {
280 |     style: "expanded",
281 |     quietDeps: true,
282 |     // Bootstrap 5 foloseste inca API-uri Sass marcate pentru o versiune
283 |     // viitoare; avertismentele sunt ascunse, fara a ascunde
... fragment scurtat; functia completa ramane in fisier ...

```

Intrebari probabile si raspunsuri

De ce browserul nu incarca SCSS?

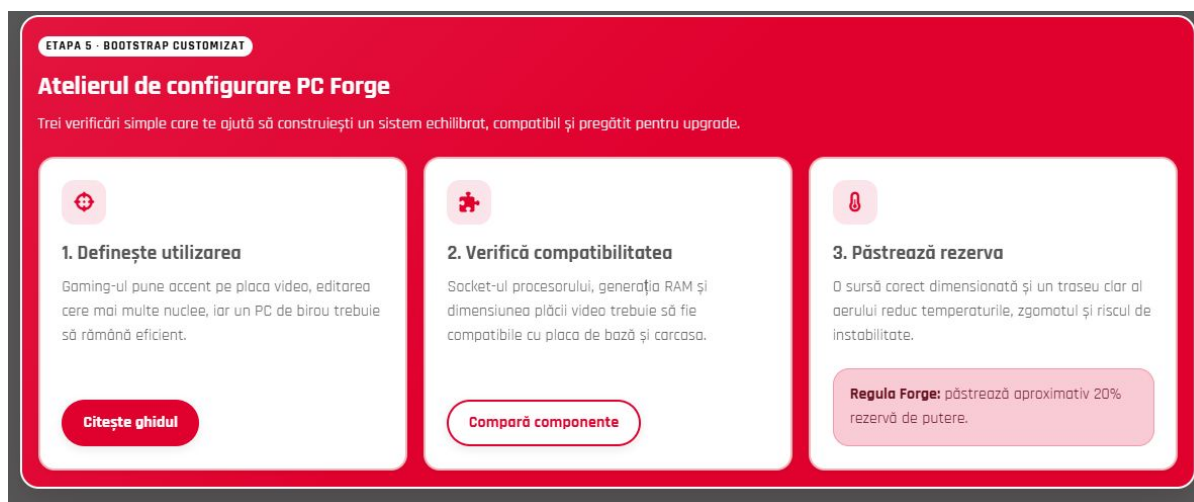
Raspuns: Browserele inteleg CSS, nu sintaxa Sass. Sass este compilat inainte.

Ce este debounce?

Raspuns: O intarziere care combina evenimente rapide intr-o singura compilare.

Ce se intampla daca Sass este invalid?

Raspuns: Compilatorul raporteaza eroarea; backupul pastreaza versiunea CSS anterioara.



Bootstrap customizat in schema PC Forge - componente reale, nu continut placeholder.

5.3 Bootstrap customizat prin Sass (0,25 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/#bootstrap-custom> | VS Code: `resurse/sass/custom-bootstrap.scss`; `resurse/css/custom-bootstrap.css`; `views/pagini/index.ejs`; `views/fragmente/head.ejs`

Ce cere exact: Bootstrap trebuie compilat cu variabile schimbate pentru culori, font, breakpointuri, raze, headinguri si alte componente, apoi folosit efectiv in pagina.

Demonstratia, in ordinea corecta

Arata cardurile PC Forge, butonul plin, butonul outline si alerta.

Redimensioneaza pagina pentru a demonstra gridul Bootstrap.

In SCSS indica variabilele declarate inaintea importului Bootstrap.

In head arata ca Bootstrap este incarcat inaintea CSS-ului proiectului.

Cum functioneaza in proiect

Variabilele Sass sunt evaluate la compilare. De aceea valorile PC Forge trebuie definite inainte de importul Bootstrap.

primary este rosul proiectului, secondary este griul, fontul este Rajdhani, breakpointurile si border-radius sunt personalizate.

Clasele row, col, card, btn si alert demonstreaza componente reale Bootstrap. Continutul explica tema PC Forge, nu este un placeholder tehnic.

CSS-ul Bootstrap este incarcat primul, iar stilurile locale vin dupa el pentru corectii precise.

Algoritmul pe care trebuie sa il intelegi

Suprascrie variabilele Sass.

Importa sursa Bootstrap.

Compilerul genereaza clasele cu noile valori.

EJS aplica acele clase componentelor.

CSS-ul local repara numai conflictele cu stilurile vechi.

Bucata-cheie: variabilele care personalizeaza frameworkul:

```

1 | /* Etapa 5 - Bootstrap customizat prin SCSS
2 | Variabilele Bootstrap sunt suprascrise inainte de importul frameworkului.
3 | Astfel, Bootstrap este generat direct cu tema PC Forge. */
4 |
5 | /* Culori principale din schema cromatica */
6 | $primary: #e1012f;
7 | $secondary: #575757;
8 | $danger: #e01816;
9 | $light: #ffffff;
10 | $dark: #1f1f1f;
11 |
12 | /* Fundal si culoare text */
13 | $body-bg: #575757;
14 | $body-color: #ffffff;
15 |
16 | /* Font implicit */
17 | $font-family-base: "Rajdhani", Arial, sans-serif;
18 |
19 | /* Dimensiuni headinguri */
20 | $h1-font-size: 3rem;
21 | $h2-font-size: 2.2rem;
22 | $h3-font-size: 1.6rem;
23 |
24 | /* Raze pentru border */
25 | $border-radius: 0.8rem;
26 | $border-radius-sm: 0.5rem;
27 | $border-radius-lg: 1.2rem;
28 | $border-radius-xl: 1.5rem;
29 |
30 | /* Breakpoint-uri custom pentru ecrane medii si mari */
31 | $grid-breakpoints: (
32 |   xs: 0,
33 |   sm: 576px,
34 |   md: 768px,
35 |   lg: 1100px,
36 |   xl: 1320px,
37 |   xxl: 1500px,
38 | );
39 |
40 | /* Latimi custom pentru containere Bootstrap */
41 | $container-max-widths: (
42 |   sm: 540px,
43 |   md: 720px,
44 |   lg: 1040px,
45 |   xl: 1240px,
46 |   xxl: 1440px,
47 | );
48 |
49 | /* Alte variabile Bootstrap modificate */
50 | $btn-border-radius: 999px;
51 | $btn-padding-x: 1.2rem;
52 | $btn-padding-y: 0.55rem;
53 |
54 | $card-border-radius: 1rem;

```



```

55 | $card-border-color: rgba(#e02f35, 0.35);
56 | $card-bg: #ffffff;
57 | $card-color: #575757;
58 |
59 | $alert-border-radius: 1rem;
60 | $alert-padding-x: 1rem;
61 | $alert-padding-y: 0.8rem;
62 |
63 | /* Import Bootstrap dupa modificarea variabilelor */
64 | @import "../node_modules/bootstrap/scss/bootstrap";

```

Intrebari probabile si raspunsuri

De ce primul buton este plin si al doilea outline?

Raspuns: Sunt doua stari Bootstrap intentionate: btn-primary este actiunea principala, iar btn-outline-primary este una secundara. Hoverul completeaza fundalul celui outline.

De ce variabilele sunt inaintea importului?

Raspuns: Bootstrap isi genereaza regulile folosind valorile existente in momentul importului.

Bootstrap inlocuieste CSS-ul proiectului?

Raspuns: Nu. Bootstrap ofera componente si utilitare; CSS-ul PC Forge pastreaza identitatea si rezolva conflictele locale.

5.4 Galeria animata (0,50 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/#galerie-animata> | VS Code: index.js; views/fragmente/galerie-animata.ejs; resurse/sass/galerie-animata.scss

Ce cere exact: La fiecare incarcare serverul alege aleator un numar de imagini divizibil cu 3 si mai mic de 16, apoi o secventa consecutiva si distincta. Sass genereaza animatia, iar hover o pune pe pauza.

Demonstratia, in ordinea corecta

Reincarca pagina si observa ca selectia se poate schimba.

Urmareste ciclul animatiei si faptul ca ultima imagine continua natural spre prima.

Tine cursorul peste galerie si verifica pauza.

Micsoreaza fereastra si arata ca galeria dispare pe ecranele unde cerinta o solicita.

Cum functioneaza in proiect

Aleatoriul nu este complet liber: lista valorilor permise contine numai multipli de 3 sub 16 si care incap in JSON.

Offsetul maxim este lungimea listei minus numarul ales. slice produce imagini consecutive fara duplicate.

EJS transmite numarul ca variabila CSS. Sass foloseste @for pentru a genera keyframes compatibile cu numarul efectiv de imagini.

animation-play-state: paused opreste timpul animatiei pe hover, nu reseteaza pozitia.

Algoritmul pe care trebuie sa il intelegi

Construieste numerele eligibile.

Alege aleator un numar si un offset valid.

Extrage secventa cu slice.

EJS creeaza elementele galeriei.

CSS muta imaginile prin cadre; hover comuta starea pe paused.

Bucata-cheie: selectia aleatoare controlata:

```

493 | function obtineGalerieAnimata() {
494 |   const imaginiDisponibile = obGlobal.obGalerie.imagini.filter(function (image) {
495 |     return image.cale_thumbnail;
496 |   });
497 |   const numerePermise = [3, 6, 9, 12].filter(function (numar) {
498 |     return numar < 16 && numar <= imaginiDisponibile.length;
499 |   });
500 |   const numarImagini =
501 |     numerePermise[Math.floor(Math.random() * numerePermise.length)];
502 |   const offset = Math.floor(Math.random() * imaginiDisponibile.length);
503 |   const imagini = [];
504 |
505 |   for (let index = 0; index < numarImagini; index++) {
506 |     imagini.push(imaginiDisponibile[(offset + index) % imaginiDisponibile.length]);
507 |   }
508 |
509 |   return { imagini: imagini, numarImagini: numarImagini };
510 | }
511 |
512 | // Etapa 4 - task 13:
513 | // initErori() citeste si parseaza erori.json, seteaza calea absoluta (servita de
514 | // server) pentru fiecare imagine folosind cale_baza si salveaza obiectul in
515 | // obGlobal.obErori
516 | /**
517 |  * Etapa 4 - Bonus: cauta proprietati JSON repetate direct in textul fisierului,
518 |  * inainte ca JSON.parse() sa pastreze numai ultima lor valoare.
519 |  * @param {string} continut Textul JSON original.

```

Intrebari probabile si raspunsuri

De ce numarul trebuie divizibil cu 3?

Raspuns: Este o constrangere explicita a cerintei si permite impartirea coerenta a secventei animate.

Cum garantezi imagini distincte?

Raspuns: slice extrage pozitii diferite din acelasi vector, fara a alege de mai multe ori aceeasi pozitie.

De ce generezi CSS cu Sass?

Raspuns: Numarul imaginilor variaza, iar @for produce automat cadrele corespunzatoare.

5.5 Backup timestamp, nume cu puncte si validarea galeriei (0,125 p)

Ce trebuie sa deschizi: Browser: Terminalul si folderul backup/resurse/css | VS Code: index.js; resurse/json/galerie.json; backup/resurse/css

Ce cere exact: Cele trei bonusuri cer copii CSS cu timp in nume, tratarea corecta a numelor cu mai multe puncte si verificarea datelor galeriei la pornire.

Demonstratia, in ordinea corecta

Arata un nume de backup care contine data si ora.

Indica path.parse si expresia care schimba numai extensia finala .scss.

Reporneste serverul si arata mesajul ca galeria a fost validata.

Explica verbal ca o imagine lipsa produce o eroare clara inainte de pornirea normala.

Cum functioneaza in proiect

Timestampul permite pastrarea mai multor versiuni fara suprascriere.

path.parse intelege separat folderul, numele si extensia; astfel tema.dark.scss devine tema.dark.css, nu tema.css.

valideazaGalerie verifica existenta folderului, structura listei si fiecare fisier mentionat.

Validarea timpurie transforma o imagine sparta greu de urmarit intr-un mesaj de pornire precis.

Algoritmul pe care trebuie sa il intelegi

Citeste si parseaza JSON-ul.

Verifica tipul proprietatilor si calea galeriei.

Construieste calea absoluta pentru fiecare imagine.

Acumuleaza sau arunca erori explicite.

Continua initializarea numai daca toate resursele sunt valide.

Bucata-cheie: validarea datelor si a fisierelor:

```

401 | function valideazaGalerie() {
402 |   const erori = [];
403 |   const caleGalerie = path.join(__dirname, String(obGlobal.obGalerie.cale_galerie ||
    "").replace(/^\//, ""));
404 |   if (!fs.existsSync(caleGalerie) || !fs.statSync(caleGalerie).isDirectory()) {
405 |     erori.push(`Folderul cale_galerie nu exista: ${caleGalerie}`);
406 |   } else {
407 |     for (const imagine of obGlobal.obGalerie.imagini || []) {
408 |       const caleImagine = path.join(caleGalerie, imagine.cale_imagine || "");
409 |       if (!imagine.cale_imagine || !fs.existsSync(caleImagine)) {
410 |         erori.push(`Imagine declarata in galerie.json, dar absenta de pe disc:
    ${caleImagine}`);
411 |       }
412 |     }
413 |   }
414 |   if (erori.length) erori.forEach((mesaj) => console.error(`[Etapa 5 - Bonus 5]
    ${mesaj}`));
415 |   else console.log("[Etapa 5 - Bonus 5] Datele galeriei au fost validate cu succes.");
416 |   return erori.length === 0;
417 | }
418 |
419 | // Miniaturile au aceeasi dimensiune si sunt decupate automat cu Sharp. Un fisier
420 | // este refacut numai daca lipseste sau daca imaginea originala este mai noua.
421 | /**
422 |  * Genereaza miniaturile WebP lipsa sau invecchite cu Sharp.
423 |  * @returns {Promise<void>} Promise rezolvat dupa procesarea imaginilor.
424 |  */
425 | async function pregatesteMiniaturiGalerie() {
426 |   const folderOriginale = path.join(
427 |     __dirname,
428 |     "resurse",
429 |     "imagini",
430 |     "galerie",
431 |     "originale",
432 |   );
433 |   const folderMiniaturi = path.join(
434 |     __dirname,
435 |     "resurse",
436 |     "imagini",
437 |     "galerie",
438 |     "thumbnails",
439 |   );
440 |
441 |   creeazaFolderDacaNuExista(folderMiniaturi);
442 |
443 |   for (const imagine of obGlobal.obGalerie.imagini) {
444 |     const caleOriginal = path.join(folderOriginale, imagine.cale_imagine);
445 |     const numeMiniatura = path.parse(imagine.cale_imagine).name + ".webp";
446 |     const caleMiniatura = path.join(folderMiniaturi, numeMiniatura);
447 |
448 |     if (!fs.existsSync(caleOriginal)) {
449 |       console.warn("Imagine lipsa din galerie: " + caleOriginal);
450 |       continue;
451 |     }

```

```
452 |  
453 | const trebuieGenerata =  
454 | !fs.existsSync(caleMiniatura) ||  
455 | fs.statSync(caleOriginal).mtimeMs > fs.statSync(caleMiniatura).mtimeMs;
```

Intrebari probabile si raspunsuri

De ce verifici la pornire?

Raspuns: Problema este descoperita imediat si serverul nu livreaza o pagina partial defecta.

De ce nu folosesti split('.')?

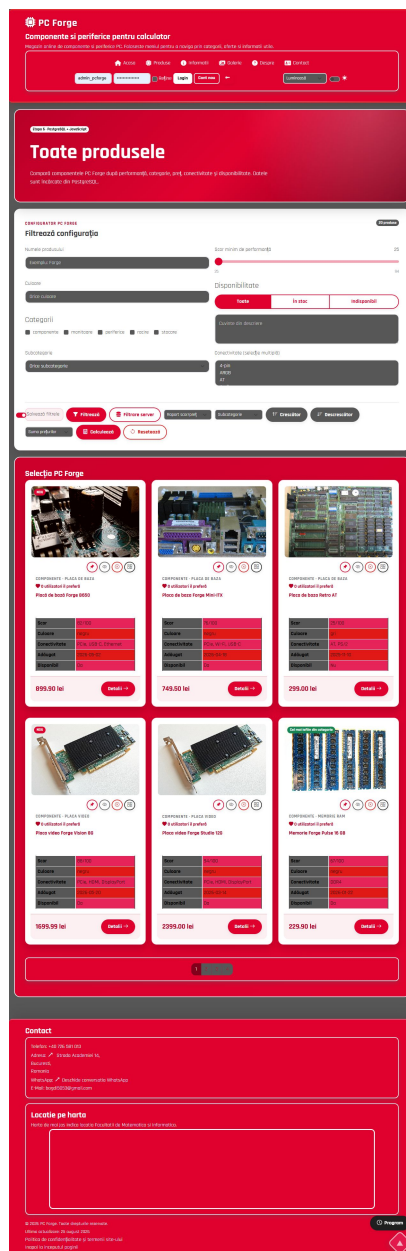
Raspuns: Un nume poate contine mai multe puncte; path.parse lucreaza corect cu ultima extensie.

Timestampul este doar decorativ?

Raspuns: Nu. El face numele unic si permite pastrarea istoricului.

ETAPA 6 - cerinte principale: 1,75 puncte

Ideea etapei: Produsele vin din PostgreSQL, sunt randate semantic cu EJS si apoi filtrate, sortate si calculate in browser. Bootstrap organizeaza controalele, iar tema se pastreaza.



Pagina /produse - filtre generate, controale Bootstrap si carduri provenite din baza de date.

6.1 Datele si paginile produselor (parte din 1,20 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> si [/produs/1](http://localhost:8080/produs/1) | VS Code: `database/*.sql`; `module/acces-produse.js`; `index.js`; `views/pagini/produse.ejs`; `views/pagini/produs.ejs`

Ce cere exact: Produsele trebuie pastrate in PostgreSQL, listate pe pagina generala si afisate individual pe o ruta care primeste id-ul produsului.

Demonstratia, in ordinea corecta

Deschide `/produse` si indica date diferite provenite din tabel.

Apasa Detalii pentru un produs si observa URL-ul `/produs/id`.

Schimba manual id-ul cu unul inexistent si arata tratarea erorii.

În cod urmărește ruta, funcția de acces la date și `res.render`.

Cum funcționează în proiect

Modulul `acces-produse` separă SQL-ul de rutele HTTP. Query-urile folosesc `$1`, `$2` pentru valori, reducând riscul de SQL injection.

Ruta `/produse` cere vectorul și îl transmite în `res.render`. EJS repetă articolul pentru fiecare produs.

Ruta `/produs/:id` transformă parametrul în număr întreg pozitiv, caută un singur produs și randează `produs.ejs` sau 404.

`data-*` de pe carduri reprezintă puntea dintre datele introduse de EJS și algoritmul JavaScript din browser.

Algoritmul pe care trebuie să îl înțelegi

Browserul cere ruta.

Express validează parametrul și apelează modulul de date.

PostgreSQL întoarce rows.

Serverul completează proprietățile calculate.

EJS produce HTML-ul trimis browserului.

Bucata-cheie: acces parametrizat la produsele listei și produsul unic:

```

35 | async function obtineProduse(categorie) {
36 |   const parametri = [];
37 |   let conditie = "";
38 |
39 |   if (categorie) {
40 |     parametri.push(categorie);
41 |     conditie = "WHERE categorie = $1::categorie_produș";
42 |   }
43 |
44 |   const rezultat = await pool.query(
45 |     `SELECT id, nume, descriere, imagine, categorie::text, subcategorie,
46 |     pret::float8 AS pret, scor_performanta,
47 |     to_char(data_adaugare, 'YYYY-MM-DD') AS data_adaugare,
48 |     culoare::text, conectivitate, in_stoc
49 |     FROM produse
50 |     ${conditie}
51 |     ORDER BY id`,
52 |     parametri,
53 |   );
54 |
55 |   return rezultat.rows;
56 | }
57 |
58 | // Etapa 6 - cerinta individuala, punctul 2: pagina unui produs unic.
59 | /**
60 |  * @param {number} id Identificatorul numeric al produsului.
61 |  * @returns {Promise<object|null>} Produsul gasit sau null.
62 |  */
63 | async function obtineProdusDupaId(id) {
64 |   const rezultat = await pool.query(
65 |     `SELECT id, nume, descriere, imagine, categorie::text, subcategorie,
66 |     pret::float8 AS pret, scor_performanta,
67 |     to_char(data_adaugare, 'YYYY-MM-DD') AS data_adaugare,
68 |     culoare::text, conectivitate, in_stoc
69 |     FROM produse
70 |     WHERE id = $1`,
71 |     [id],
72 |   );
73 |   return rezultat.rows[0] || null;
74 | }
75 |
76 | // Etapa 6 - Bonus 16: produse similare din aceeasi categorie, cu prioritate
77 | // pentru aceeasi subcategorie si fara produsul curent.
78 | /** @param {object} produs Produsul de referinta. @param {number} [limita=4] Limita.
79 |  * @returns {Promise<object[]>} Produse similare. */
80 | async function obtineProduseSimilare(produs, limita = 4) {
81 |   const rezultat = await pool.query(
82 |     `SELECT id, nume, descriere, imagine, categorie::text, subcategorie,
83 |     pret::float8 AS pret, scor_performanta

```

Intrebari probabile si raspunsuri

Ce este o ruta dinamica?

Raspuns: O ruta precum /produs/:id, unde :id este o valoare extrasa din URL.

Ce face await?

Raspuns: Suspenda functia async pana cand Promise-ul query-ului se rezolva, fara a bloca intregul server.

De ce separi acces-produse.js?

Raspuns: Ruta se ocupa de HTTP, iar modulul de date de SQL; fiecare responsabilitate poate fi verificata separat.

6.2 Cele opt filtre si validarea (parte din 1,20 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: `views/pagini/produse.ejs`; `resurse/js/produse.js`

Ce cere exact: Filtrele folosesc text, range, datalist, radio, checkbox, textarea, select simplu si select multiplu. Valorile nepotrivite trebuie ascunse, nu sterse din DOM, iar inputurile afectate trebuie validate.

Demonstratia, in ordinea corecta

Testeaza pe rand numele, scorul, culoarea, stocul, categoria, descrierea, subcategoria si conectivitatea.

Combina doua filtre si explica faptul ca toate conditiile sunt legate prin AND.

Introdu doua caractere in textarea si arata starea is-invalid; la trei caractere eroarea dispare.

Apasa Resetare si confirma revenirea la ordinea si valorile initiale.

Cum functioneaza in proiect

`obtainStareFiltre` citeste toate controalele si creeaza un singur obiect coerent.

`aplicaFiltrare` parcurge cardurile deja existente si compara valorile inputurilor cu dataset-ul produsului.

`includes` este folosit pentru text, comparatia numerica pentru scor, iar `every` cere ca toate optiunile de conectivitate selectate sa existe la produs.

`hidden` ascunde cardul, dar il pastreaza in DOM pentru resetare si filtrari ulterioare.

`valideazaInputuri` foloseste expresii regulate, `setCustomValidity`, `is-invalid` si `reportValidity`.

Algoritmul pe care trebuie sa il intelegi

Citeste si normalizeaza valorile filtrelor.

Verifica validitatea campurilor.

Pentru fiecare produs calculeaza opt conditii booleene.

Combina conditiile cu operatorul `&&`.

Salveaza eligibilitatea, apoi actualizeaza paginarea, mesajul si contorul.

Bucata-cheie: cele opt conditii ale filtrarii:

```

121 | function aplicaFiltrare(afiseazaMesaje = false) {
122 |   valoareScor.value = inpScor.value;
123 |   if (!valideazaInputuri(afiseazaMesaje)) return false;
124 |
125 |   const filtru = {
126 |     nume: normalizeazaText(inpNume.value.trim()),
127 |     scor: Number(inpScor.value),
128 |     culoare: inpCuloare.value.trim().toLowerCase(),
129 |     stoc: document.querySelector('input[name="stoc"]:checked').value,
130 |     categorii: valoriSelectate(".filtru-categorie"),
131 |     descriere: normalizeazaText(inpDescriere.value.trim()),
132 |     subcategorie: selectSubcategorie.value,
133 |     conectivitate: optiuniMultiple(selectConectivitate),
134 |   };
135 |
136 |   let numarVizibile = 0;
137 |   produse.forEach(function (produs) {
138 |     const conexiuniProdus = JSON.parse(produs.dataset.conectivitate);
139 |     const corespunde =
140 |       normalizeazaText(produs.dataset.nume).includes(filtru.nume) &&
141 |       Number(produs.dataset.scor) >= filtru.scor &&
142 |       (!filtru.culoare || produs.dataset.culoare === filtru.culoare) &&
143 |       (filtru.stoc === "toate" || (filtru.stoc === "da" && produs.dataset.stoc === "true")
144 |       || (filtru.stoc === "nu" && produs.dataset.stoc === "false")) &&
145 |       (filtru.categorii.length === 0 ||
146 |       filtru.categorii.includes(produs.dataset.categorie)) &&
147 |       normalizeazaText(produs.dataset.descriere).includes(filtru.descriere) &&
148 |       (!filtru.subcategorie || produs.dataset.subcategorie === filtru.subcategorie) &&
149 |       (filtru.conectivitate.length === 0 || filtru.conectivitate.every((valoare) =>
150 |       conexiuniProdus.includes(valoare)));
151 |     // Etapa 6 - Bonus 6: un produs fixat suprascrie filtrul, iar cel ascuns
152 |     // in sessionStorage ramane invizibil numai in tabul curent.
153 |     produs.dataset.ascunsTemporar = "false";
154 |     const eligibil = (corespunde || produs.classList.contains("produs-card--fixat")) &&
155 |     !ascunseTab.has(produs.dataset.produsId);
156 |     produs.dataset.corespunde = String(eligibil);
157 |     if (eligibil) numarVizibile++;
158 |   });
159 |   // Etapa 6 - Bonus 3 si Bonus 15.
160 |   mesajFaraProduse.hidden = numarVizibile !== 0;
161 |   numarProduse.textContent = `${numarVizibile} ${numarVizibile === 1 ? "produs" :
162 |   "produse"}`;
163 |   paginaCurenta = 1;
164 |   actualizeazaPaginare();
165 |   salveazaStareFi
... fragment scurtat; functia completa ramane in fisier ...

```

Intrebari probabile si raspunsuri

De ce folosesti dataset?

Raspuns: EJS pune datele produsului in atribute data-*, iar JavaScript le poate citi fara o noua cerere la server.

De ce hidden si nu remove?

Raspuns: remove ar elimina nodul; hidden permite readucerea lui la resetare.

Validarea din browser este suficienta pentru baza de date?

Raspuns: Nu. Este pentru experienta utilizatorului; datele trimise serverului trebuie validate din nou.

6.3 Sortarea, calculul si resetarea (parte din 1,20 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [resurse/js/produse.js](#); [views/pagini/produse.ejs](#)

Ce cere exact: Sortarea foloseste doua chei, calculul opereaza numai pe produsele vizibile, iar resetarea cere confirmare si reface ordinea initiala.

Demonstratia, in ordinea corecta

Alege pret si nume, apoi sorteaza crescator si descrescator.

Alege suma, media, minim sau maxim si arata mesajul care dispare dupa doua secunde.

Aplica filtre, apoi apasa Resetare si raspunde OK.

Arata revenirea cheilor, filtrelor si cardurilor la starea initiala.

Cum functioneaza in proiect

Comparatorul intoarce diferenta primei chei. Numai la egalitate compara a doua cheie.

sens este 1 pentru crescator si -1 pentru descrescator, evitand duplicarea algoritmului.

append muta nodurile existente in noua ordine; nu le cloneaza.

Calculul filtreaza cardurile neascunse, transforma preturile in Number si foloseste reduce, Math.min sau Math.max.

indexInitial pastreaza pozitia initiala necesara resetarii.

Algoritmul pe care trebuie sa il intelegi

Valideaza inputurile.

Construieste un vector nou din produse.

Sorteaza dupa cheia principala, apoi secundara.

Muta cardurile in DOM.

Pentru calcul extrage preturile vizibile si creeaza mesajul cu createElement.

La resetare sterge starile persistente si sorteaza dupa indexInitial.

Bucata-cheie: comparatorul, calculul si inceputul resetarii:

```

190 | function valoareSortare(produs, cheie) {
191 |   if (cheie === "raport") return Number(produs.dataset.scor) /
Number(produs.dataset.pret);
192 |   if (cheie === "pret" || cheie === "scor") return Number(produs.dataset[cheie]);
193 |   return normalizeazaText(produs.dataset[cheie]);
194 | }
195 |
196 | function comparaValori(a, b) {
197 |   if (typeof a === "number" && typeof b === "number") return a - b;
198 |   return String(a).localeCompare(String(b), "ro", { numeric: true });
199 | }
200 |
201 | // Etapa 6 - punctul 8.2 si Bonus 8: sortare dupa doua chei alese.
202 | function sorteazaProduce(sens) {
203 |   if (!valideazaInputuri(true)) return;
204 |   const cheie1 = document.getElementById("cheie-sortare-1").value;
205 |   const cheie2 = document.getElementById("cheie-sortare-2").value;
206 |   const sortate = [...produse].sort(function (produsA, produsB) {
207 |     const primaComparatie = comparaValori(valoareSortare(produsA, cheie1),
valoareSortare(produsB, cheie1));
208 |     if (primaComparatie !== 0) return primaComparatie * sens;
209 |     return comparaValori(valoareSortare(produsA, cheie2), valoareSortare(produsB,
cheie2)) * sens;
210 |   });
211 |   sortate.forEach((produs) => lista.append(produs));
212 |   produse.forEach((produs) => { produs.dataset.ascunsTemporar = "false"; });
213 |   actualizeazaPaginare();
214 | }
215 |
216 | // Etapa 6 - punctul 8.3: rezultatul este creat dinamic si sters dupa 2 secunde.
217 | function calculeazaPreturi() {
218 |   if (!valideazaInputuri(true)) return;
219 |   const preturi = produse.filter((produs) => !produs.hidden).map((produs) =>
Number(produs.dataset.pret));
220 |   if (!preturi.length) {
221 |     afiseazaCalcul("Nu există produse pentru calcul.");
222 |     return;
223 |   }
224 |   const tip = document.getElementById("tip-calcul").value;
225 |   const suma = preturi.reduce((total, pret) => total + pret, 0);
226 |   const rezultate = { suma: suma, medie: suma / preturi.length, minim:
Math.min(...preturi), maxim: Math.max(...preturi) };
227 |   const etichete = { suma: "Suma", medie: "Media", minim: "Minimul", maxim: "Maximul" };
228 |   afiseazaCalcul(`${etichete[tip]} prețurilor: ${rezultate[tip].toFixed(2)} lei`);
229 | }
230 |
231 | function afiseazaCalcul(text) {
232 |   document.querySelectorAll(".rezultat-calcul").forEach((element) => element.remove());
233 |
... fragment scurtat; functia completa ramane in fisier ...

```

Intrebari probabile si raspunsuri

De ce comparatorul poate intoarce un numar negativ?

Raspuns: Negativ inseamna ca primul element trebuie pus inainte, pozitiv dupa, iar zero inseamna egalitate.

Ce face reduce?

Raspuns: Combina toate valorile intr-un acumulator; aici aduna preturile.

De ce rezultatul dispare?

Raspuns: Cerinta cere un element creat dinamic, vizibil doua secunde si apoi sters cu setTimeout.

6.4 Stilizarea Bootstrap a inputurilor (0,30 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: `views/pagini/produse.ejs`; `resurse/sass/custom-bootstrap.scss`; `resurse/css/custom-bootstrap.css`

Ce cere exact: Butoanele si inputurile folosesc Bootstrap, textarea are floating label si stare invalida, radio/checkbox au aspect de toggle, controalele sunt asezate in grid, tema foloseste un control Bootstrap, iar range-ul este personalizat prin variabile Sass.

Demonstratia, in ordinea corecta

Indica butoanele cu iconuri si micsoreaza fereastra pentru a ramane doar iconurile.

Arata floating label-ul textarea si provoaca starea invalida.

Apasa radio-urile de stoc si observa trecerea outline-plin.

Arata coloanele Bootstrap si bulina marita a range-ului.

Cum functioneaza in proiect

form-control, form-select, form-range si btn sunt clase Bootstrap, nu reguli imitate manual.

btn-check ascunde vizual inputul, dar labelul conectat prin for devine buton toggle accesibil.

row si col-* definesc coloanele la breakpointuri.

Variabilele Sass pentru range schimba dimensiunea, culoarea thumbului si pista inaintea compilarii Bootstrap.

Algoritmul pe care trebuie sa il intelegi

EJS produce controale cu clasele frameworkului.

Bootstrap aplica aspectul de baza si gridul.

JavaScript adauga is-invalid cand valoarea nu respecta regula.

CSS-ul customizat furnizeaza paleta si dimensiunile PC Forge.

Bucata-cheie: cele opt controale si butoanele Bootstrap:

```

23 | <div class="row g-3" id="zona-filtre">
24 | <div class="col-12 col-md-6 col-xl-4">
25 | <label class="form-label" for="inp-nume">Numele produsului</label>
26 | <input class="form-control" type="text" id="inp-nume" placeholder="Exemplu: Forge"
   | autocomplete="off" />
27 | <div class="invalid-feedback">Folosește numai litere, spații și cratimă.</div>
28 | </div>
29 |
30 | <div class="col-12 col-md-6 col-xl-4">
31 | <label class="form-label d-flex justify-content-between" for="inp-scor"><span>Scor
   | minim de performanță</span><output id="valoare-scor" for="inp-scor"><%= filtre.scorMinim
   | %></output></label>
32 | <!-- Etapa 6 - cerinta 6b: min/max si valoarea selectata sunt generate din date. -->
33 | <input class="form-range" type="range" id="inp-scor" min="<%= filtre.scorMinim %>"
   | max="<%= filtre.scorMaxim %>" value="<%= filtre.scorMinim %>" />
34 | <div class="range-limite"><span><%= filtre.scorMinim %></span><span><%=
   | filtre.scorMaxim %></span></div>
35 | </div>
36 |
37 | <div class="col-12 col-md-6 col-xl-4">
38 | <label class="form-label" for="inp-culoare">Culoare</label>
39 | <input class="form-control" type="text" id="inp-culoare" list="lista-culori"
   | placeholder="Orice culoare" autocomplete="off" />
40 | <datalist id="lista-culori"><% filtre.culori.forEach(function(culoare) { %><option
   | value="<%= culoare %>"></option><% }>; %></datalist>
41 | <div class="invalid-feedback">Alege o culoare existentă din listă.</div>
42 | </div>
43 |
44 | <fieldset class="col-12 col-md-6 col-xl-4">
45 | <legend class="form-label">Disponibilitate</legend>
46 | <!-- Etapa 6 - task Bootstrap 0.3c: radio buttons ca toggle buttons. -->
47 | <div class="btn-group w-100" role="group" aria-label="Disponibilitate">
48 | <input class="btn-check" type="radio" name="stoc" id="stoc-toate" value="toate" checked
   | /><label class="btn btn-outline-primary" for="stoc-toate">Toate</label>
49 | <input class="btn-check" type="radio" name="stoc" id="stoc-da" value="da" /><label
   | class="btn btn-outline-primary" for="stoc-da">În stoc</label>
50 | <input class="btn-check" type="radio" name="stoc" id="stoc-nu" value="nu" /><label
   | class="btn btn-outline-primary
... fragment scurtat; functia completa ramane in fisier ...

```

Intrebari probabile si raspunsuri

Cum dovedesti ca este Bootstrap?

Raspuns: Arat clasele form-control, form-range, btn-check, row, col si btn, plus variabilele compilate din Sass.

De ce labelul trebuie legat de input?

Raspuns: Clickul pe label activeaza controlul, iar tehnologiile asistive cunosc numele campului.

De ce textul butoanelor dispare pe mobil?

Raspuns: Iconul pastreaza actiunea recognoscibila si economiseste spatiu; textul ramane accesibil semantic.

6.5 Tema light/dark persistenta (0,20 p)

Ce trebuie sa deschizi: Browser: Orice pagina; selectorul de tema din header | VS Code: resurse/js/tema.js; views/fragmente/header.ejs; resurse/css/teme.css

Ce cere exact: Utilizatorul schimba tema, iar alegerea se pastreaza in localStorage si se aplica pe paginile urmatoare.

Demonstratia, in ordinea corecta

Alege tema intunecata.

Navigheaza pe alta pagina si apoi reincarca.

Arata ca tema ramane activa.

In DevTools sau cod indica atributul temei si cheia din localStorage.

Cum functioneaza in proiect

localStorage pastreaza perechi cheie-valoare chiar dupa inchiderea paginii.

Scriptul citeste preferinta devreme si seteaza atributul/clasa documentului.

CSS foloseste acel selector pentru a schimba variabilele de culoare fara a dubla structura HTML.

Algoritmul pe care trebuie sa il intelegi

Citeste tema salvata sau foloseste valoarea implicita.

Aplica tema pe elementul document.

La schimbarea controlului reaplica tema.

Salveaza numele temei pentru vizitele urmatoare.

Bucata-cheie: citirea, aplicarea si persistenta temei:

```

1 | // Etapa 6 - task tema light/dark:
2 | // tema se memoreaza in localStorage si se aplica pe toate paginile site-ului.
3 | (function () {
4 |   const cheieTema = "pc-forge-tema";
5 |   const temaServer = window.pcForgeUtilizator?.tema;
6 |   const temaSalvata = temaServer || localStorage.getItem(cheieTema);
7 |   // Etapa 6 - Bonus 2: sunt acceptate trei teme persistente.
8 |   const temaInitiala = ["light", "dark", "contrast"].includes(temaSalvata) ? temaSalvata :
   "light";
9 |
10 |   function aplicaTema(tema) {
11 |     document.documentElement.dataset.tema = tema;
12 |     const comutator = document.getElementById("comutator-tema");
13 |     const selector = document.getElementById("selector-tema");
14 |     if (selector) selector.value = tema;
15 |     if (comutator) {
16 |       comutator.checked = tema === "dark";
17 |       comutator.setAttribute("aria-label", tema === "dark" ? "Activează tema luminoasă" :
   "Activează tema întunecată");
18 |     }
19 |   }
20 |
21 |   aplicaTema(temaInitiala);
22 |   const comutator = document.getElementById("comutator-tema");
23 |   const selector = document.getElementById("selector-tema");
24 |   function memoreazaTema(tema) {
25 |     localStorage.setItem(cheieTema, tema);
26 |     aplicaTema(tema);
27 |     if (window.pcForgeUtilizator) fetch("/api/tema", { method: "POST", headers: {
   "Content-Type": "application/json" }, body: JSON.stringify({ tema: tema }) });
28 |   }
29 |   if (comutator) {
30 |     comutator.addEventListener("change", function () {
31 |       const tema = comutator.checked ? "dark" : "light";
32 |       memoreazaTema(tema);
33 |     });
34 |   }
35 |   if (selector) selector.addEventListener("change", () =>
   memoreazaTema(selector.value));
36 | })();

```

Intrebari probabile si raspunsuri

localStorage ajunge automat la server?

Raspuns: Nu. Ramane in browser. Pentru preferinta contului exista separat salvarea prin server.

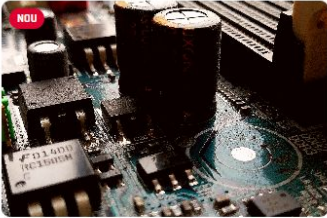
De ce folosesti variabile CSS?

Raspuns: Aceeasi componenta isi schimba culorile prin valori comune, fara reguli duplicate pentru fiecare element.

ETAPA 6 - bonusurile vizuale: 4,05 puncte

Ideea etapei: Aceste bonusuri extind acelasi sistem de produse. Invata functia aplicaFiltrare foarte bine: ea explica simultan filtrarea imediata, mesajul, contorul, paginarea si produsele fixate.

Selecția PC Forge



NOU

★

🔄

🔍

📄

COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori il preferă

Placă de bază Forge B650

Scor	82/100
Culoare	negru
Conectivitate	PCIe, USB-C, Ethernet
Adăugat	2026-05-02
Disponibil	Da

899.90 lei

Detalii →



★

🔄

🔍

📄

COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori il preferă

Placa de baza Forge Mini-ITX

Scor	76/100
Culoare	negru
Conectivitate	PCIe, Wi-Fi, USB-C
Adăugat	2026-04-18
Disponibil	Da

749.50 lei

Detalii →



★

🔄

🔍

📄

COMPONENTE - PLACA DE BAZA

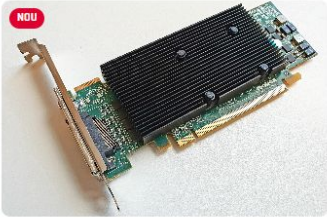
♥ 0 utilizatori il preferă

Placa de baza Retro AT

Scor	25/100
Culoare	gri
Conectivitate	AT, PS/2
Adăugat	2025-11-10
Disponibil	Nu

299.00 lei

Detalii →



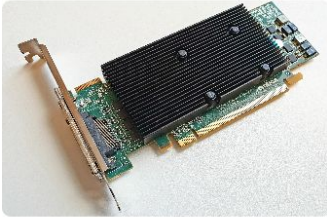
NOU

★

🔄

🔍

📄

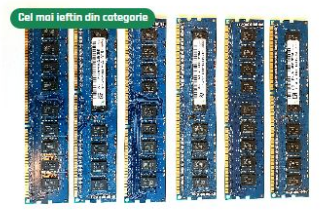


★

🔄

🔍

📄



Cel mai ieftin din categorie

★

🔄

🔍

📄

Cardurile contin date, marcaje si actiuni folosite de mai multe bonusuri.

6.B2 Trei teme persistente (0,15 p)

Ce trebuie sa deschizi: Browser: Selectorul din header, pe / si /produse | VS Code: resurse/js/tema.js; resurse/css/teme.css; views/fragmente/header.ejs

Ce cere exact: Alegerea nu se limiteaza la light/dark: proiectul ofera Luminoasa, Intunecata si Contrast Forge, toate persistente.

Demonstratia, in ordinea corecta

Comuta succesiv cele trei teme.

Pentru fiecare, indica schimbarea contrastului si a suprafetelor.

Lasa Contrast Forge activa, reincarca si navigheaza.

Cum functioneaza in proiect

Bonusul extinde mecanismul obligatoriu; cheia localStorage stocheaza numele uneia dintre valorile permise.

Tema Contrast Forge nu este doar inversarea culorilor, ci o a treia configuratie definita explicit.

Algoritmul pe care trebuie sa il intelegi

Valideaza alegerea in lista temelor permise.

Actualizeaza documentul.

Persistenta pastreaza alegerea.

Bucata-cheie: aceeasi functie gestioneaza toate cele trei teme:

```

1 | // Etapa 6 - task tema light/dark:
2 | // tema se memoreaza in localStorage si se aplica pe toate paginile site-ului.
3 | (function () {
4 |   const cheieTema = "pc-forge-tema";
5 |   const temaServer = window.pcForgeUtilizator?.tema;
6 |   const temaSalvata = temaServer || localStorage.getItem(cheieTema);
7 |   // Etapa 6 - Bonus 2: sunt acceptate trei teme persistente.
8 |   const temaInitiala = ["light", "dark", "contrast"].includes(temaSalvata) ? temaSalvata :
   "light";
9 |
10 |   function aplicaTema(tema) {
11 |     document.documentElement.dataset.tema = tema;
12 |     const comutator = document.getElementById("comutator-tema");
13 |     const selector = document.getElementById("selector-tema");
14 |     if (selector) selector.value = tema;
15 |     if (comutator) {
16 |       comutator.checked = tema === "dark";
17 |       comutator.setAttribute("aria-label", tema === "dark" ? "Activează tema luminoasă" :
   "Activează tema întunecată");
18 |     }
19 |   }
20 |
21 |   aplicaTema(temaInitiala);
22 |   const comutator = document.getElementById("comutator-tema");
23 |   const selector = document.getElementById("selector-tema");
24 |   function memoreazaTema(tema) {
25 |     localStorage.setItem(cheieTema, tema);
26 |     aplicaTema(tema);
27 |     if (window.pcForgeUtilizator) fetch("/api/tema", { method: "POST", headers: {
   "Content-Type": "application/json" }, body: JSON.stringify({ tema: tema }) });
28 |   }
29 |   if (comutator) {
30 |     comutator.addEventListener("change", function () {
31 |       const tema = comutator.checked ? "dark" : "light";
32 |       memoreazaTema(tema);
33 |     });
34 |   }
35 |   if (selector) selector.addEventListener("change", () =>
   memoreazaTema(selector.value));
36 | })();

```

Intrebari probabile si raspunsuri

Este acelasi punctaj ca light/dark?

Raspuns: Nu. Light/dark este cerinta de baza de 0,20; a treia tema aduce bonusul separat de 0,15.

6.B3+B15 Mesajul fara rezultate si contorul (0,10 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [resurse/js/produse.js](#); [views/pagini/produse.ejs](#)

Ce cere exact: Dupa fiecare filtrare se afiseaza numarul produselor eligibile; la zero, lista este inlocuita vizual de un mesaj relevant.

Demonstratia, in ordinea corecta

Observa numarul initial.

Scrie un nume imposibil si arata zero plus mesajul.

Sterge textul si arata revenirea numarului.

Cum functioneaza in proiect

numarVizibile este incrementat in aceeaasi bucla ce evalueaza filtrarea, deci nu se parcurge inutil lista din nou.

hidden este comutat pe mesaj in functie de comparatia cu zero, iar textContent evita interpretarea continutului ca HTML.

Algoritmul pe care trebuie sa il intelegi

Porneste contorul de la zero.

Incrementeaza pentru fiecare produs eligibil.

Actualizeaza textul singular/plural.

Ascunde sau afiseaza mesajul.

Bucata-cheie: numararea si actualizarea celor doua elemente:

```

145 | normalizeazaText(produs.dataset.descriere).includes(filtru.descriere) &&
146 | (!filtru.subcategorie || produs.dataset.subcategorie === filtru.subcategorie) &&
147 | (filtru.conectivitate.length === 0 || filtru.conectivitate.every((valoare) =>
conexiuniProdus.includes(valoare)));
148 |
149 | // Etapa 6 - Bonus 6: un produs fixat suprascrie filtrul, iar cel ascuns
150 | // in sessionStorage ramane invizibil numai in tabul curent.
151 | produs.dataset.ascunsTemporar = "false";
152 | const eligibil = (corespunde || produs.classList.contains("produs-card--fixat")) &&
!ascunseTab.has(produs.dataset.produsId);
153 | produs.dataset.corespunde = String(eligibil);
154 | if (eligibil) numarVizibile++;
155 | });
156 |
157 | // Etapa 6 - Bonus 3 si Bonus 15.
158 | mesajFaraProduce.hidden = numarVizibile !== 0;
159 | numarProduce.textContent = `${numarVizibile} ${numarVizibile === 1 ? "produs" :
"produse"}`;
160 | paginaCurenta = 1;
161 | actualizeazaPaginare();
162 | salveazaStareFiltre();
163 | return true;
164 | }
165 |
166 | // Etapa 6 - Bonus 5: genereaza ceil(N/K) butoane si afiseaza intervalul paginii P.

```

Intrebari probabile si raspunsuri

De ce textContent si nu innerHTML?

Raspuns: Aici avem text simplu; textContent evita parsarea HTML si este alegerea mai sigura.

Produsul este sters?

Raspuns: Nu, doar ascuns. Poate reveni imediat dupa schimbarea filtrului.

6.B4 Filtrarea imediata (maximum 0,40 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [resurse/js/produse.js](#)

Ce cere exact: Fiecare dintre cele opt filtre reaplica algoritmul imediat, la input pentru text/range/textarea si la change pentru controalele discrete.

Demonstratia, in ordinea corecta

Tasteaza lent in nume si observa actualizarea la fiecare caracter.

Muta range-ul si urmareste scorul si cardurile.

Bifeaza categoria si radio-ul fara a apasa Filtreaza.

Cum functioneaza in proiect

input se declanseaza in timpul tastarii sau deplasarii range-ului. change se potriveste selecturilor, radio-urilor si checkboxurilor.

matches alege evenimentul eficient pentru fiecare control. Callbackul comun apeleaza aplicaFiltrare(false).

Bonusul depinde de functionarea corecta a celor opt filtre; nu este suficienta doar atasarea evenimentului.

Algoritmul pe care trebuie sa il intelegi

Selecteaza toate controalele din zona filtrelor.

Pentru fiecare determina tipul.

Ataseaza input sau change.

La eveniment ruleaza validarea discreta si filtrarea.

Bucata-cheie: atasarea evenimentului potrivit tuturor filtrelor:

```
356 | salveazaFiltre.addEventListener("change", function () {
357 |   if (salveazaFiltre.checked) salveazaStareFiltre();
358 |   else localStorage.removeItem(cheieFiltrePersistente);
359 | });
360 |
361 | // Etapa 6 - Bonus 4: filtrarea se actualizeaza imediat la schimbarea inputurilor.
362 | document.querySelectorAll("#zona-filtre input, #zona-filtre textarea, #zona-filtre
select").forEach(function (input) {
363 |   const eveniment = input.matches('input[type="text"], input[type="range"], textarea') ?
"input" : "change";
364 |   input.addEventListener(eventiment, () => aplicaFiltrare(false));
365 | });
366 | restaureazaStareFiltre();
367 | aplicaFiltrare();
368 | afiseazaComparatie();
369 | animeazaCarduriBootstrap();
370 | });
```

Intrebari probabile si raspunsuri

De ce nu folosesti click peste tot?

Raspuns: Click nu descrie toate modificarile, de exemplu tastarea sau selectia cu tastatura.

De ce false la aplicaFiltrare?

Raspuns: Filtrarea live nu trebuie sa afiseze agresiv mesaje native la fiecare tasta; butoanele explicite pot folosi true.

6.B5 Paginarea (0,50 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [resurse/js/produse.js](#); [views/pagini/produse.ejs](#)

Ce cere exact: Pentru N produse si K=6 produse pe pagina se genereaza $\text{ceil}(N/K)$ butoane, iar pagina P afiseaza intervalul corect.

Demonstratia, in ordinea corecta

Reseteaza filtrele pentru a avea suficiente produse.

Apasa pagina 2 si indica schimbarea celor sase carduri.

Aplica un filtru si arata recalcularea numarului de pagini.

În cod indica `Math.ceil` și `slice`.

Cum functionează în proiect

`Math.ceil` rotunjește în sus deoarece și un rest de produs necesită o pagină nouă.

Indicii intervalului sunt $(P-1)*K$ inclusiv și $P*K$ exclusiv, exact convenția `slice`.

Butoanele sunt create dinamic cu `createElement`; pagina activă primește clasa Bootstrap `active`.

Paginarea se aplică după filtrare, asupra produselor eligibile, nu asupra întregii liste.

Algoritmul pe care trebuie să îl înțelegi

Construiești vectorul eligibile.

Calculează numărPagini cu `Math.ceil`.

Limitează pagina curentă la noul număr.

Ascunde toate cardurile și afișează slice-ul paginii.

Reconstruiești butoanele și ascultătorii lor.

Bucata-cheie: formula și intervalul paginii:

```
167 | function actualizeazaPaginare() {
168 |   const eligibile = produse.filter((produs) => produs.dataset.corespunde === "true" &&
produs.dataset.ascunsTemporar !== "true");
169 |   const numarPagini = Math.max(1, Math.ceil(eligibile.length / produsePePagina));
170 |   paginaCurenta = Math.min(paginaCurenta, numarPagini);
171 |   produse.forEach((produs) => { produs.hidden = true; });
172 |   eligibile.slice((paginaCurenta - 1) * produsePePagina, paginaCurenta *
produsePePagina).forEach((produs) => { produs.hidden = false; });
173 |   const container = document.getElementById("paginare-produse");
174 |   container.replaceChildren();
175 |   if (numarPagini <= 1) return;
176 |   for (let pagina = 1; pagina <= numarPagini; pagina++) {
177 |     const element = document.createElement("li");
178 |     element.className = `page-item${pagina === paginaCurenta ? " active" : ""}`;
179 |     const buton = document.createElement("button");
180 |     buton.className = "page-link";
181 |     buton.type = "button";
182 |     buton.textContent = pagina;
183 |     buton.setAttribute("aria-label", `Pagina ${pagina}`);
184 |     buton.addEventListener("click", () => { paginaCurenta = pagina;
actualizeazaPaginare(); document.getElementById("titlu-lista-produse").scrollIntoView({
behavior: "smooth" }); });
185 |     element.append(buton);
186 |     container.append(element);
187 |   }
188 | }
189 |
190 | function valoareSortare(produs, cheie) {
```

Întrebări probabile și răspunsuri

De ce `Math.max(1, ...)`?

Răspuns: Pastrează o stare internă validă chiar dacă nu există rezultate; mesajul separat explică lista vidă.

De ce nu incarci o pagina noua?

Raspuns: Aceasta implementare face paginare client-side, permisa de cerinta deoarece toate produsele sunt deja in DOM.

Ce face slice?

Raspuns: Returneaza elementele dintre indexul de inceput inclusiv si cel final exclusiv.

6.B6 Fixare si doua tipuri de ascundere (0,50 p)

Ce trebuie sa deschizi: Browser: `http://localhost:8080/produse` | VS Code: `resurse/js/produse.js`; `views/pagini/produse.ejs`

Ce cere exact: Fiecare card are trei butoane cu icon si tooltip: pastreaza la filtrare, ascunde pana la urmatoarea filtrare si ascunde in tab prin sessionStorage.

Demonstratia, in ordinea corecta

Fixeaza un produs si aplica un filtru care in mod normal l-ar exclude.

Ascunde temporar alt produs, apoi reaplica filtrarea si arata ca poate reveni.

Ascunde un produs pentru tab, reincarca si arata ca ramane ascuns.

Deschide pagina in alt tab si arata ca produsul exista acolo.

Cum functioneaza in proiect

Fixarea foloseste o clasa CSS; expresia corespunde OR fixat face exceptia explicita.

Ascunderea temporara foloseste dataset si se reseteaza la filtrare/sortare.

Ascunderea de tab salveaza id-ul in sessionStorage. Aceasta stocare este separata pentru fiecare tab si dispare cand tabul se inchide.

Delegarea evenimentului pune un singur listener pe lista si gaseste butonul/cardul cu closest.

Algoritmul pe care trebuie sa il intelegi

Intercepta clickul pe lista.

Gaseste cardul si actiunea apasata.

Pentru fixare comuta clasa si stilul butonului.

Pentru ascunderea simpla marcheaza dataset.

Pentru tab adauga id-ul in Set si serializeaza-l in sessionStorage.

Recalculeaza filtrarea sau paginarea.

Bucata-cheie: cele trei ramuri ale aceluiasi listener:


```

259 | produse.forEach((produs) => { produs.classList.remove("produs-card--fixat");
produs.dataset.ascunsTemporar = "false"; });
260 | aplicaFiltrare();
261 | }
262 |
263 | // Etapa 6 - Bonus 6: cele trei actiuni cerute pentru fiecare card.
264 | lista.addEventListener("click", function (eveniment) {
265 |   const card = eveniment.target.closest(".produs-card");
266 |   if (!card) return;
267 |   const fixare = eveniment.target.closest(".actiune-fixare");
268 |   const ascundere = eveniment.target.closest(".actiune-ascundere");
269 |   const sesiune = eveniment.target.closest(".actiune-sesiune");
270 |   if (fixare) {
271 |     card.classList.toggle("produs-card--fixat");
272 |     fixare.classList.toggle("btn-primary",
card.classList.contains("produs-card--fixat"));
273 |     fixare.classList.toggle("btn-outline-primary",
!card.classList.contains("produs-card--fixat"));
274 |     aplicaFiltrare();
275 |   } else if (ascundere) {
276 |     card.dataset.ascunsTemporar = "true";
277 |     actualizeazaPaginare();
278 |   } else if (sesiune) {
279 |     ascunseTab.add(card.dataset.produsId);
280 |     sessionStorage.setItem(cheieAscunseTab, JSON.stringify([...ascunseTab]));
281 |     aplicaFiltrare();
282 |   }
283 | });
284 |
285 | // Etapa 6 - Bonus 11: click pe zona neinteractiva a cardului deschide modalul.
286 | lista.addEventListener("click", function (eveniment) {
287 |   if (eveniment.target.closest("a, button, input, select, label")) return;
288 |   const card = eveniment.target.closest(".produs-card");
289 |   if (!card || !window.bootstrap) return;

```

Intrebari probabile si raspunsuri

localStorage si sessionStorage sunt identice?

Raspuns: Nu. localStorage persista intre sesiuni; sessionStorage apartine tabului curent.

De ce Set?

Raspuns: Set pastreaza id-uri unice si are verificare rapida prin has.

Ce este event delegation?

Raspuns: Ascultam pe parinte si identificam elementul copil care a declansat actiunea.

6.B7 Cautarea fara diacritice (0,15 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [resurse/js/produse.js](#)

Ce cere exact: Textul introdus fara diacritice trebuie sa gaseasca valori care contin diacritice si invers.

Demonstratia, in ordinea corecta

Cauta un termen existent mai intai cu diacritice, apoi fara.

Arata ca textul afisat nu este modificat.

In cod indica normalize NFD si expresia semnelor combinante.

Cum functioneaza in proiect

`toLocaleLowerCase('ro')` uniformizeaza majusculele in context romanesc.

`normalize('NFD')` descompune litera accentuata in litera de baza plus marcaj combinant.

Expresia `[\u0300-\u036f]` elimina marcatele; inlocuirile pentru `s` si `t` acopera forme Unicode care pot ramane.

Se normalizeaza copii folosite la comparatie, nu continutul vizibil.

Algoritmul pe care trebuie sa il intelegi

Converteste la litere mici.

Descompune Unicode.

Elimina diacriticele combinante.

Compara textele normalizate cu `includes`.

Bucata-cheie: functia mica reutilizata de mai multe filtre:

```
40 | function normalizeazaText(text) {
41 |   return text.toLocaleLowerCase("ro").normalize("NFD").replace(/[\u0300-\u036f]/g,
    |   "").replace(/ș/g, "s").replace(/ț/g, "t");
42 | }
43 |
44 | // Etapa 6 - punctul 10: validarea se executa inaintea operatiilor.
```

Intrebari probabile si raspunsuri

Ce este Unicode?

Raspuns: Standardul care atribuie coduri caracterelor; aceeași litera vizuala poate avea reprezentari compuse diferit.

Modifici numele produsului?

Raspuns: Nu. Transformarea este folosita exclusiv pentru comparatie.

6.B8 Sortarea dupa doua chei (0,35 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [resurse/js/produse.js](#); [views/pagini/produse.ejs](#)

Ce cere exact: Utilizatorul alege doua chei dintre cel putin trei si sensul. Cheia secundara este folosita numai cand valorile primei chei sunt egale.

Demonstratia, in ordinea corecta

Alege pret ca prima cheie si nume ca a doua.

Sorteaza crescator si identifica primele rezultate.

Schimba prima cheie cu scor sau raport si sorteaza descrescator.

In cod indica primaComparatie si ramura de egalitate.

Cum functioneaza in proiect

valoareSortare intoarce Number pentru pret/scor/raport si text normalizat pentru nume/categorie.

comparaValori trateaza separat numerele si textele cu localeCompare.

Operatorul sens controleaza crescator/descrescator. Acelasi comparator este reutilizat.

Copierea [...produse] evita modificarea referintei colectiei de baza inaintea reordonarii DOM.

Algoritmul pe care trebuie sa il intelegi

Citeste cheile din selecturi.

Calculeaza prima comparatie.

Daca este nenula, intoarce rezultatul inmultit cu sens.

Daca este zero, calculeaza cheia a doua.

Muta nodurile sortate in lista.

Bucata-cheie: comparatorul cu departajare:

```

190 | function valoareSortare(produs, cheie) {
191 | if (cheie === "raport") return Number(produs.dataset.scor) /
    Number(produs.dataset.pret);
192 | if (cheie === "pret" || cheie === "scor") return Number(produs.dataset[cheie]);
193 | return normalizeazaText(produs.dataset[cheie]);
194 | }
195 |
196 | function comparaValori(a, b) {
197 | if (typeof a === "number" && typeof b === "number") return a - b;
198 | return String(a).localeCompare(String(b), "ro", { numeric: true });
199 | }
200 |
201 | // Etapa 6 - punctul 8.2 si Bonus 8: sortare dupa doua chei alese.
202 | function sorteazaProduse(sens) {
203 | if (!valideazaInputuri(true)) return;
204 | const cheie1 = document.getElementById("cheie-sortare-1").value;
205 | const cheie2 = document.getElementById("cheie-sortare-2").value;
206 | const sortate = [...produse].sort(function (produsA, produsB) {
207 | const primaComparatie = comparaValori(valoareSortare(produsA, cheie1),
    valoareSortare(produsB, cheie1));
208 | if (primaComparatie !== 0) return primaComparatie * sens;
209 | return comparaValori(valoareSortare(produsA, cheie2), valoareSortare(produsB,
    cheie2)) * sens;
210 | });
211 | sortate.forEach((produs) => lista.append(produs));
212 | produse.forEach((produs) => { produs.dataset.ascunsTemporar = "false"; });
213 | actualizeazaPaginare();
214 | }
215 |
216 | // Etapa 6 - punctul 8.3: rezultatul este creat dinamic si sters dupa 2 secunde.

```

Intrebari probabile si raspunsuri

De ce ai nevoie de cheia a doua?

Raspuns: Oferă o ordine determinată când mai multe produse au aceeași valoare principală.

Ce face localeCompare?

Raspuns: Compară texte conform regulilor unei limbi și poate trata natural numerele incluse în siruri.

6.B9 Imagini multiple pentru produs (0,50 p)

Ce trebuie să deschizi: Browser: <http://localhost:8080/produs/1> | VS Code: module/imagini-produse.js; index.js; views/pagini/produs.ejs; resurse/imagini/produse-variante

Ce cere exact: Produsul are imagine principală și minimum două variante care pot fi schimbate printr-un carusel Bootstrap.

Demonstratia, in ordinea corecta

Deschide un produs și folosește controalele caruselului.

Indica faptul că toate cadrele aparțin aceluiași produs.

În folder arată fișierele -detaliu.webp și -complet.webp.

În modul arata Sharp și funcția `obtainImaginiProdus`.

Cum funcționează în proiect

La pornire pregătește `Variante` parcurge imaginile unice ale produselor. Sharp redimensionează și exportă WebP.

Variantele sunt derivate din imaginea produsului curent; nu se folosesc fotografii ale produselor similare.

`obtainImaginiProdus` construiește lista original plus două cai derivate. Ruta o pune în `produs.imagini`, iar EJS construiește `carousel-item`.

WebP reduce dimensiunea, iar Sharp automatizează generarea coerentă.

Algoritmul pe care trebuie să îl înțelegi

Determină sursa originală.

Construiește două destinații stabile.

Generează variantele dacă lipsesc sau trebuie actualizate.

Ruta cere lista imaginilor pentru produsul curent.

EJS marchează prima imagine activă și Bootstrap gestionează navigarea.

Bucata-cheie: generarea și returnarea celor trei cai:

```

1 | // Etapa 6 - Bonus 9: imagini multiple deduse din calea imaginii principale.
2 | const fs = require("fs");
3 | const path = require("path");
4 | const sharp = require("sharp");
5 |
6 | const folderProiect = path.join(__dirname, "..");
7 | const folderOriginale = path.join(folderProiect, "resurse", "imagini", "galerie",
  "originale");
8 | const folderVariante = path.join(folderProiect, "resurse", "imagini",
  "produse-variante");
9 | const extensiiAcceptate = new Set([".jpg", ".jpeg", ".png", ".webp"]);
10 |
11 | /**
12 |  * Construiește numele public al unei variante pornind din imaginea principală.
13 |  * @param {string} calePrincipala Calea salvată în tabela produse.
14 |  * @param {string} sufix Tipul cadrului generat.
15 |  * @returns {string} Calea HTTP a imaginii derivate.
16 |  */
17 | function caleVarianta(calePrincipala, sufix) {
18 |   const baza = path.parse(String(calePrincipala)).name;
19 |   return `./resurse/imagini/produse-variante/${baza}-${sufix}.webp`;
20 | }
21 |
22 | /**
23 |  * Etapa 6 - Bonus 9: generează pentru fiecare fotografie un cadru de detaliu și
24 |  * unul complet. Fisierul este refăcut numai dacă originalul este mai nou.
25 |  * @returns {Promise<void>} Promisiune rezolvată după generarea variantelor.
26 |  */
27 | async function pregatesteVariante() {
28 |   fs.mkdirSync(folderVariante, { recursive: true });
29 |   const fisiere = fs.readdirSync(folderOriginale).filter((nume) =>
30 |     extensiiAcceptate.has(path.extname(nume).toLowerCase()),
31 |   );
32 |
33 |   for (const nume of fisiere) {
34 |     const sursa = path.join(folderOriginale, nume);
35 |     const baza = path.parse(nume).name;
36 |     const variante = [
37 |       {
38 |         destinatie: path.join(folderVariante, `${baza}-detaliu.webp`),
39 |         transforma: (pipeline) => pipeline.resize(900, 650, { fit: "cover", position:
  "attention" }),
40 |       },
41 |       {
42 |         destinatie: path.join(folderVariante, `${baza}-complet.webp`),
43 |         transforma: (pipeline) => pipeline.resize(900, 650, { fit: "contain", background:
  "#f4f4f6" }),
44 |       },
45 |     ];
46 |
47 |     for (const varianta of variante) {
48 |       const trebuieGenerata =
49 |         !fs.existsSync(varianta.destinatie) ||
50 |         fs.statSync(sursa).mtimeMs > fs.statSync(varianta.destinatie).mtimeMs;

```

```
51 | if (trebuieGenerata) {
52 |
... fragment scurtat; functia completa ramane in fisier ...
```

Intrebari probabile si raspunsuri

De ce nu folosesti imaginile similare?

Raspuns: Cerinta cere imagini multiple ale aceluiasi produs; acelea ar reprezenta alte produse.

Ce este Sharp?

Raspuns: O biblioteca Node pentru procesarea imaginilor: redimensionare, decupare si conversie.

De ce generezi la pornire?

Raspuns: Aplicatia se asigura automat ca variantele necesare exista inainte ca pagina sa le ceara.

6.B11 Modalul Bootstrap (0,40 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [views/pagini/produse.ejs](#); [resurse/js/produse.js](#); [resurse/css/custom-bootstrap.css](#)

Ce cere exact: Clickul pe zona neinteractiva a cardului deschide un modal tematizat cu datele celui produs si posibilitate de inchidere.

Demonstratia, in ordinea corecta

Apasa pe suprafata libera a unui card, nu pe un buton.

Arata titlul, imaginea, descrierea, categoria, scorul si pretul.

Inchide cu X, cu butonul si prin click in exterior.

Apasa Pagina produsului si arata linkul dinamic.

Cum functioneaza in proiect

Listenerul este delegat listei. Guard-ul ignora linkurile, butoanele si inputurile pentru a nu deschide accidental modalul.

Datele sunt citite din dataset si introduse in elementele modalului.

`getOrCreateInstance` foloseste componenta JavaScript oficiala Bootstrap si reutilizeaza instanta.

Linkul este construit cu id-ul produsului curent.

Algoritmul pe care trebuie sa il intelegi

Gaseste cel mai apropiat card.

Opreste actiunea daca tinta este interactiva.

Completeaza titlul si continutul.

Seteaza href-ul produsului.

Obtine instanta Bootstrap si apeleaza `show`.

Bucata-cheie: popularea si deschiderea modalului:

```

291 | document.getElementById("modal-produs-continut").innerHTML = `<p>${card.dataset.descriere}</p><dl
class="row mb-0"><dt class="col-5">Categorie</dt><dd
class="col-7">${card.dataset.categorie}</dd><dt class="col-5">Performanță</dt><dd
class="col-7">${card.dataset.scor}/100</dd><dt class="col-5">Preț</dt><dd
class="col-7">${Number(card.dataset.pret).toFixed(2)} lei</dd></dl>`;
292 | document.getElementById("modal-produs-link").href =
`/produs/${card.dataset.produsId}`;
293 |
window.bootstrap.Modal.getOrCreateInstance(document.getElementById("modal-produs")).show();
294 | });
295 |
296 | // Etapa 6 - Bonus 20: comparatia persista cel mult o zi in localStorage.
297 | function citesteComparatie() {
298 |   try {
299 |     const stare = JSON.parse(localStorage.getItem(cheieComparatie) || "null");
300 |     if (!stare || Date.now() - stare.actualizat > 24 * 60 * 60 * 1000) return [];
301 |     return Array.isArray(stare.produce) ? stare.produce.slice(0, 2) : [];
302 |   } catch { return []; }
303 | }
304 | function scrieComparatie(selectie) {

```

Intrebari probabile si raspunsuri

De ce nu scrii propriul modal de la zero?

Raspuns: Bonusul cere componenta Bootstrap; folosesc API-ul oficial pentru focus, fundal si inchidere.

Ce este getOrCreateInstance?

Raspuns: Returneaza instanta existenta pentru element sau creeaza una daca nu exista.

6.B14 Cel mai ieftin produs din fiecare categorie (0,30 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: index.js; views/pagini/produse.ejs

Ce cere exact: Pentru fiecare categorie se calculeaza pretul minim si produsele corespunzatoare primesc un marcaj clar.

Demonstratia, in ordinea corecta

Gaseste insigna Cel mai ieftin.

Compara pretul cu alt produs din aceeasi categorie.

In index.js arata Map-ul si cele doua treceri.

Cum functioneaza in proiect

Map asociaza fiecare categorie cu minimul curent. Prima parcurgere construiește minimele.

A doua parcurgere seteaza esteCelMaileftin comparand pretul produsului cu minimul categoriei.

Calculul se face pe server, iar EJS decide afisarea badge-ului.

Algoritmul pe care trebuie sa il intelegi

Porneste Map gol.

Pentru fiecare produs inlocuieste minimul daca pretul este mai mic.

Parcurge din nou produsele.

Marcheaza egalitatea cu minimul categoriei.

EJS afiseaza badge-ul conditionat.

Bucata-cheie: calculul minimului si proprietatea booleana:

```
803 | const pretMinimPeCategorie = new Map();
804 | produse.forEach(function (produs) {
805 |   const minimCurent = pretMinimPeCategorie.get(produs.categorie);
806 |   if (minimCurent === undefined || produs.pret < minimCurent) {
807 |     pretMinimPeCategorie.set(produs.categorie, produs.pret);
808 |   }
809 | });
810 | // Etapa 6 - Bonus 12d: pret redus calculat fara modificarea bazei.
811 | const ofertaCurenta = oferte.obtineOfertaCurenta();
812 | produse.forEach((produs) => {
813 |   if (ofertaCurenta && produs.categorie === ofertaCurenta.categorie) {
814 |     produs.oferta = ofertaCurenta;
815 |     produs.pretRedus = produs.pret * (1 - ofertaCurenta.reducere / 100);
816 |   }
817 | });
818 | const acum = Date.now();
819 | const intervalProdusNou = 120 * 24 * 60 * 60 * 1000;
820 | produse.forEach(function (produs) {
821 |   produs.esteCelMaiIeftin =
822 |     produs.pret === pretMinimPeCategorie.get(produs.categorie);
823 |   produs.esteNou =
824 |     acum - new Date(produs.data_adaugare + "T00:00:00").getTime() <=
825 |     intervalProdusNou;
826 | });
827 |
```

Intrebari probabile si raspunsuri

De ce doua treceri?

Raspuns: La prima descoperim minimul fiecarei categorii; abia apoi putem marca toate produsele care il egaleaza.

Ce este Map?

Raspuns: O colectie de perechi cheie-valoare; aici cheia este categoria si valoarea este pretul minim.

6.B16 Produsele similare (0,30 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produs/1> | VS Code: module/acces-produse.js; index.js; views/pagini/produs.ejs

Ce cere exact: Pagina produsului afiseaza recomandari selectate din baza dupa un criteriu clar si exclude produsul curent.

Demonstratia, in ordinea corecta

Deschide un produs si coboara la Produse similare.

Arata categoria produsului curent si a recomandarilor.

Apasa o recomandare si observa noul id din URL.

In SQL indica excluderea id-ului curent si LIMIT.

Cum functioneaza in proiect

Criteriul ales este categoria comuna. Query-ul parametrizat primeste categoria si id-ul.

Conditia `id <> $2` evita recomandarea paginii curente catre ea insasi.

LIMIT pastreaza zona scurta. EJS afiseaza informatii sumare si linkuri catre rutele produselor.

Algoritmul pe care trebuie sa il intelegi

Primeste obiectul produs curent.

Executa SELECT pentru aceeasi categorie.

Exclude id-ul curent.

Limiteaza numarul de randuri.

Transmite vectorul separat template-ului.

Bucata-cheie: interogarea produselor similare:

```

80 | const rezultat = await pool.query(
81 | `SELECT id, nume, descriere, imagine, categorie::text, subcategorie,
82 | pret::float8 AS pret, scor_performanta
83 | FROM produse
84 | WHERE id <> $1 AND categorie = $2::categorie_produc
85 | ORDER BY (subcategorie = $3) DESC, ABS(pret - $4), id
86 | LIMIT $5`,
87 | [produs.id, produs.categorie, produs.subcategorie, produs.pret, limita],
88 | );
89 | return rezultat.rows;
90 | }
91 |
92 | // Etapa 6 - Bonus 20: citirea stricta a celor doua produse cerute de comparatie.
93 | /** @param {number[]} ids Identificatori validati. @returns {Promise<object[]>}
    Produsele comparate. */
94 | async function obtineProduseDupaIduri(ids) {
95 | const rezultat = await pool.query(
96 | `SELECT id, nume, descriere, imagine, categorie::text, subcategorie,
97 | pret::float8 AS pret, scor_performanta, culoare::text,
98 | conectivitate, in_stoc
99 | FROM produse WHERE id = ANY($1::int[]) ORDER BY array_position($1::int[], id)`,
100 | [ids],
101 | );
102 | return rezultat.rows;
103 | }
104 |
105 | // Etapa 6 - Bonus 17: seturile, produsele lor si pretul redus sunt calculate in SQL.
106 | /** @param {number|null} [idProdus=null] Filtrare optionala dupa produs. @returns
    {Promise<object[]>} Seturi grupate. */
107 | async function obtineSeturi(idProdus = null) {
108 | const rezultat = await pool.query(
109 | `SELECT s.id,s.nume_set,s.descriere_set,
110 | json_agg(json_build_object('id',p.id,'nume',p.nume,'imagine',p.imagine,'pret',p.pret:
    :float8) ORDER BY p.id) AS produse,
111 | SUM(p.pret)::float8 AS pret_brut, COUNT(p.id)::int AS numar_produce
112 | FROM seturi s JOIN asociere_set a ON a.id_set=s.id JOIN produse p ON p.id=a.id_produc
113 | ${idProdus ? "WHERE s.id IN (SELECT id_set FROM asociere_set WHERE id_produc=$1)" :
    ""}
114 | GROUP BY s.id ORDER BY s.id`,
115 | idProdus ? [idProdus] : [],

```

Intrebari probabile si raspunsuri

De ce categoria este un criteriu bun?

Raspuns: Produsele au acelasi tip general si sunt relevante ca alternative; criteriul este usor de explicat si verificat.

De ce folosesti LIMIT?

Raspuns: Nu incarc inutil toata categoria si pastrez pagina usor de parcurs.

6.B18 Produsele noi (0,20 p)

Ce trebuie sa deschizi: Browser: `http://localhost:8080/` si `/produse` | VS Code: `index.js`; `module/acces-orm.js`; `views/fragmente/produse-noi.ejs`; `views/pagini/produse.ejs`

Ce cere exact: Un produs este nou daca diferenta dintre data curenta si data adaugarii este cel mult intervalul T; primeste marcaj si apare intr-o sectiune ordonata invers cronologic.

Demonstratia, in ordinea corecta

Arata badge-ul Nou in `/produse`.

Pe prima pagina arata sectiunea cu cele mai noi produse.

Apasa un link din sectiune.

In cod indica `intervalProdusNou` si ordonarea descrescatoare.

Cum functioneaza in proiect

Data din baza este transformata intr-un obiect `Date`, iar diferenta in milisecunde este comparata cu 120 de zile.

`esteNou` este o proprietate booleana calculata pe server si consumata simplu de EJS.

Sectiunea principala foloseste ordonarea dupa `data_adaugare DESC`; implementarea ORM poate fi mentionata numai daca esti intrebat.

Algoritmul pe care trebuie sa il intelegi

Obtine data curenta.

Calculeaza diferenta pentru fiecare produs.

Seteaza `esteNou`.

Ordoneaza produsele noi descrescator dupa data.

EJS afiseaza badge-ul si linkul.

Bucata-cheie: regula de timp pentru marcajul Nou:

```
815 | produs.pretRedus = produs.pret * (1 - ofertaCurenta.reducere / 100);
816 | }
817 | });
818 | const acum = Date.now();
819 | const intervalProdusNou = 120 * 24 * 60 * 60 * 1000;
820 | produse.forEach(function (produs) {
821 |   produs.esteCelMaiIeftin =
822 |   produs.pret === pretMinimPeCategorie.get(produs.categorie);
823 |   produs.esteNou =
824 |   acum - new Date(produs.data_adaugare + "T00:00:00").getTime() <=
825 |   intervalProdusNou;
826 | });
827 |
```

Intrebari probabile si raspunsuri

Ce unitate are `Date.now`?

Raspuns: Milisecunde de la epoca Unix; de aceea si intervalul este convertit in milisecunde.

De ce data vine din baza?

Raspuns: Starea de produs nou trebuie sa depinda de data reala a inregistrarii, nu de HTML scris manual.

6.B19 Orarul disponibil pe orice pagina (0,20 p)

Ce trebuie sa deschizi: Browser: Orice pagina; butonul Program din interfata | VS Code: views/fragmente/orar.ejs; resurse/js/orar.js; views/fragmente/footer.ejs sau header.ejs

Ce cere exact: Un fragment EJS reutilizabil afiseaza toate zilele, marcheaza ziua curenta, spune Deschis/Inchis si poate fi deschis fara schimbarea paginii.

Demonstratia, in ordinea corecta

Deschide orarul de pe doua pagini diferite.

Indica marcarea zilei curente si mesajul Deschis/Inchis.

Inchide containerul si arata ca nu ai schimbat ruta.

In cod arata include-ul fragmentului si calculul zilei/orei.

Cum functioneaza in proiect

Fragmentul evita duplicarea aceluiasi tabel pe fiecare pagina.

JavaScript foloseste ziua si ora curenta pentru a selecta intervalul potrivit.

Afisarea este un strat peste pagina curenta; continutul poate fi inchis sau ascuns automat.

Algoritmul pe care trebuie sa il intelegi

Include fragmentul in structura comuna.

La deschidere citeste data curenta.

Gaseste randul zilei.

Compara ora cu intervalul.

Actualizeaza marcajul si starea, apoi afiseaza containerul.

Bucata-cheie: logica de afisare si calcul a starii programului:

```

1 | // Etapa 6 - Bonus 19: panou de program fara navigare, cu stare calculata dupa
2 | // ziua si ora curenta, evidentiarea zilei si inchidere automata.
3 | function initializeazaOrar() {
4 |   const buton = document.getElementById("btn-orar");
5 |   const panou = document.getElementById("panou-orar");
6 |   const inchide = document.getElementById("inchide-orar");
7 |   const stare = document.getElementById("stare-orar");
8 |   if (!buton || !panou || !inchide || !stare) return;
9 |
10 |   let temporizator;
11 |
12 |   function actualizeazaStare() {
13 |     const acum = new Date();
14 |     const rand = panou.querySelector(`[data-zi="${acum.getDay()}"]`);
15 |     panou.querySelectorAll("tbody tr").forEach((element) =>
16 |       element.classList.remove("zi-curenta"));
17 |     if (!rand) return;
18 |     rand.classList.add("zi-curenta");
19 |
20 |     const minuteAcum = acum.getHours() * 60 + acum.getMinutes();
21 |     const inMinute = (ora) => {
22 |       const [ore, minute] = ora.split(":").map(Number);
23 |       return ore * 60 + minute;
24 |     };
25 |     const esteDeschis = rand.dataset.inchis !== "true" &&
26 |       minuteAcum >= inMinute(rand.dataset.deschidere) &&
27 |       minuteAcum < inMinute(rand.dataset.inchidere);
28 |     stare.className = `alert mb-3 ${esteDeschis ? "alert-success" : "alert-secondary"}`;
29 |     stare.textContent = esteDeschis
30 |       ? `Suntem deschisi acum, pana la ${rand.dataset.inchidere}.`
31 |       : "Suntem inchisi acum. Consulta intervalele de mai jos.";
32 |   }
33 |
34 |   function ascundePanou() {
35 |     window.clearTimeout(temporizator);
36 |     panou.hidden = true;
37 |     buton.setAttribute("aria-expanded", "false");
38 |   }
39 |
40 |   function afiseazaPanou() {
41 |     actualizeazaStare();
42 |     panou.hidden = false;
43 |     buton.setAttribute("aria-expanded", "true");
44 |     window.clearTimeout(temporizator);
45 |     temporizator = window.setTimeout(ascundePanou, 12000);
46 |   }
47 |
48 |   buton.addEventListener("click", function () {
49 |     if (panou.hidden) afiseazaPanou();
50 |     else ascundePanou();
51 |   });
52 |   inchide.addEventListener("click", ascundePanou);
53 | }

```

```
54 |  
55 | // Etapa 6 - Bonus 19: initializarea functioneaza si daca fisierul este incarcat  
56 | // dupa evenimentul DOMContentLoaded (de exemplu din cache sau dintr-un fragment).  
  
... fragment scurtat; functia completa ramane in fisier ...
```

Intrebari probabile si raspunsuri

De ce fragment EJS?

Raspuns: Aceeasi structura este reutilizata pe toate paginile si se modifica intr-un singur loc.

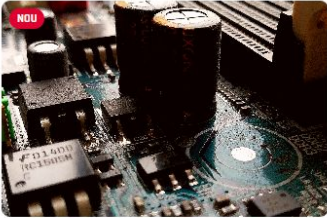
Se face o pagina noua?

Raspuns: Nu. Cerinta cere vizualizarea fara schimbarea paginii, deci containerul apare peste continutul curent.

ETAPA 7 - selectia simpla: 1 punct

Ideea etapei: Inveti doar cele trei elemente usor de demonstrat: aparitia treptata a cardurilor, bannerul cookie si filtrele persistente.

Selecția PC Forge



NOU

COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

Placă de bază Forge B650

Scor	82/100
Culoare	negru
Conectivitate	PCIe, USB-C, Ethernet
Adăugat	2026-05-02
Disponibil	Oa

899.90 lei

Detalii →



COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

Placă de bază Forge Mini-ITX

Scor	76/100
Culoare	negru
Conectivitate	PCIe, Wi-Fi, USB-C
Adăugat	2026-04-18
Disponibil	Oa

749.50 lei

Detalii →



COMPONENTE - PLACA DE BAZA

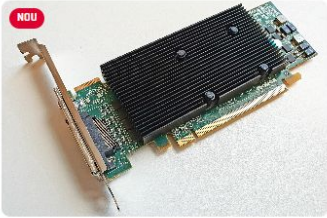
♥ 0 utilizatori îl preferă

Placă de bază Retro AT

Scor	25/100
Culoare	gri
Conectivitate	AT, PS/2
Adăugat	2025-11-10
Disponibil	Nu

299.00 lei

Detalii →



NOU

COMPONENTE - PLACA DE BAZA

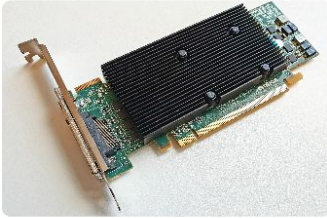
♥ 0 utilizatori îl preferă

Placă de bază Forge B650

Scor	82/100
Culoare	negru
Conectivitate	PCIe, USB-C, Ethernet
Adăugat	2026-05-02
Disponibil	Oa

899.90 lei

Detalii →



COMPONENTE - PLACA DE BAZA

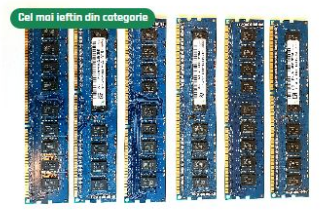
♥ 0 utilizatori îl preferă

Placă de bază Forge Mini-ITX

Scor	76/100
Culoare	negru
Conectivitate	PCIe, Wi-Fi, USB-C
Adăugat	2026-04-18
Disponibil	Oa

749.50 lei

Detalii →



Cel mai ieftin din categorie

COMPONENTE - PLACA DE BAZA

♥ 0 utilizatori îl preferă

Placă de bază Retro AT

Scor	25/100
Culoare	gri
Conectivitate	AT, PS/2
Adăugat	2025-11-10
Disponibil	Nu

299.00 lei

Detalii →

Cardurile Bootstrap sunt animate prin cooperarea dintre EJS, JavaScript și CSS.

7.1 Carduri Bootstrap afisate treptat (0,30 p)

Ce trebuie să deschizi: Browser: <http://localhost:8080/produse> | VS Code: `views/pagini/produse.ejs`; `resurse/js/produse.js`; `resurse/css/etapa7.css`

Ce cere exact: Cardurile tip Bootstrap apar la t, 2t, 3t și așa mai departe, iar durata totală depășește 3t.

Demonstratia, in ordinea corecta

Reincarca /produse și nu interacționează în primele secunde.

Urmărește apariția succesivă a cardurilor.

În cod arată $t=200$ și formula $(index+1)*t$.

În CSS arată starea inițială și clasa vizibilă.

Cum functioneaza in proiect

EJS produce toate cardurile din prima. Ele nu sunt descărcate pe rând.

JavaScript adaugă starea inițială și programează adăugarea clasei vizibile cu `setTimeout`.

CSS definește opacity/transform și transition; JavaScript stabilește numai momentul.

prefers-reduced-motion dezactivează efectul pentru utilizatorii care solicită mai puțină mișcare.

Algoritmul pe care trebuie să îl înțelegi

Colectează cardurile.

Pentru fiecare calculează întârzierea din index.

setTimeout înregistrează callbackul.

Callbackul adaugă clasa vizibilă.

CSS interpolează între stări.

Bucătărie: programarea neblocați a apariției:

```
20 | const cheieAscunseTab = "pc-forge-produse-ascunse-tab";
21 | const cheieComparatie = "pc-forge-comparatie";
22 | let paginaCurenta = 1;
23 | const ascunseTab = new Set(JSON.parse(sessionStorage.getItem(cheieAscunseTab) ||
  "[ ]"));
24 | const culoriValide = Array.from(document.querySelectorAll("#lista-culori
option")).map((optiune) => optiune.value.toLowerCase());
25 |
26 | // Etapa 7 - cerința 16 bootstrap_js: cardurile Bootstrap apar la t, 2t, 3t...
27 | // Pentru 20 de produse, ultimul apare la 4 secunde, deci animația depășește 3*t.
28 | function animeazaCarduriBootstrap() {
29 |   if (window.matchMedia("(prefers-reduced-motion: reduce)").matches) return;
30 |   const t = 200;
31 |   produse.forEach(function (produs, index) {
32 |     produs.classList.add("produs-card--animat");
33 |     window.setTimeout(function () {
34 |       produs.classList.add("produs-card--vizibil");
35 |     }, (index + 1) * t);
36 |   });
37 | }
38 |
39 | // Etapa 6 - Bonus 7: diacriticele sunt echivalente cu literele fără diacritice.
```

Întrebări probabile și răspunsuri

setTimeout blochează pagina?

Răspuns: Nu. Programează callbackul, iar firul poate continua celelalte operații.

De ce nu poți delay doar în CSS?

Răspuns: Cerința solicită realizarea în JavaScript cu setTimeout; CSS rămâne responsabil de aspectul tranziției.

Cardurile nu există până apar?

Răspuns: Există deja în DOM; inițial sunt transparente/deplasate.

7.2 Bannerul animat si cookie-ul (0,30 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080> intr-o fereastră privata | VS Code: [views/fragmente/banner-cookies.ejs](#); [resurse/js/cookies.js](#); [resurse/css/etapa7.css](#)

Ce cere exact: Bannerul porneste mic si transparent din coltul stanga-jos, se animeaza cinci secunde si ramane. Acceptarea salveaza un cookie pentru sapte zile, iar bannerul dispare si nu reapare la refresh.

Demonstratia, in ordinea corecta

Deschide o fereastră privata si incarca pagina.

Urmareste animatia completa: dimensiune, opacitate si culoare.

Apasa OK si reincarca.

Arata cookie-ul `pc_forge_cookies_acceptate` si functiile de creare/stergere.

Cum functioneaza in proiect

`citesteCookie` cauta perechea `nume=valoare` in `document.cookie` si decodeaza valoarea.

`seteazaCookie` calculeaza data expirarii, adauga `path=/` si `SameSite=Lax`.

La `DOMContentLoaded`, bannerul este pornit numai daca acceptarea lipseste. `requestAnimationFrame` asigura aplicarea starii initiale inainte de clasa animata.

Butonul OK seteaza cookie-ul si adauga starea de inchidere. Cookie-ul nu contine parola si nu este sesiunea de autentificare.

Algoritmul pe care trebuie sa il intelegi

Citeste cookie-ul de acceptare.

Daca exista, nu afisa bannerul.

Daca lipseste, elimina starea ascunsa si porneste animatia.

La OK scrie cookie-ul cu expirare sapte zile.

Ascunde bannerul; la refresh conditia initiala il exclude.

Bucata-cheie: utilitarele cookie si decizia de afisare:

```

1 | // Etapa 7 - animatie-banner si cookie-uri (cerinta individuala 0.3p).
2 | (function () {
3 | "use strict";
4 |
5 | /** Returneaza valoarea decodificata a cookie-ului cerut. */
6 | function citesteCookie(num) {
7 | const prefix = `${encodeURIComponent(num)}=`;
8 | const pereche = document.cookie.split("; ").find((element) =>
element.startsWith(prefix));
9 | return pereche ? decodeURIComponent(pereche.slice(prefix.length)) : null;
10 | }
11 |
12 | /** Salveaza un cookie pentru numarul indicat de zile. */
13 | function seteazaCookie(num, valoare, zile = 7) {
14 | const expirare = new Date(Date.now() + zile * 24 * 60 * 60 * 1000).toUTCString();
15 | document.cookie = `${encodeURIComponent(num)}=${encodeURIComponent(valoare)};
expires=${expirare}; path=/; SameSite=Lax`;
16 | }
17 |
18 | /** Etapa 7 - sterge cookie-ul primit ca parametru. */
19 | function deleteCookie(num) {
20 | document.cookie = `${encodeURIComponent(num)}=; expires=Thu, 01 Jan 1970 00:00:00 GMT;
path=/; SameSite=Lax`;
21 | }
22 |
23 | /** Etapa 7 - sterge toate cookie-urile accesibile paginii. */
24 | function deleteAllCookies() {
25 | document.cookie.split(";").forEach(function (pereche) {
26 | const num = decodeURIComponent(pereche.split("=")[0].trim());
27 | if (num) deleteCookie(num);
28 | });
29 | }
30 |
31 | // Functiile raman accesibile din consola pentru demonstratia ceruta.
32 | window.deleteCookie = deleteCookie;
33 | window.deleteAllCookies = deleteAllCookies;
34 |
35 | document.addEventListener("DOMContentLoaded", function () {
36 | const banner = document.getElementById("banner");
37 | const butonOk = document.getElementById("ok_cookies");
38 | const infoActivitate = document.getElementById("info-cookie-activitate");
39 | const cookieAcceptat = citesteCookie("pc_forge_cookies_acceptate");
40 |
41 | if (banner && !cookieAcceptat) {
42 | banner.hidden = false;
43 | requestAnimationFrame(function () {
44 | requestAnimationFrame(() => banner.classList.add("banner--vizibil"));
45 | });
46 | }
47 |
48 | if (butonOk) {
49 | butonOk.addEventListener("click", function () {
50 | seteazaCookie("pc_forge_cookies_acceptate", "da", 7);
51 | banner.classList.add("banner--inchis");

```

```
52 | window.setTimeout
... fragment scurtat; functia completa ramane in fisier ...
```

Intrebari probabile si raspunsuri

Cookie si localStorage sunt acelasi lucru?

Raspuns: Nu. Cookie-ul are expirare si este trimis in cererile compatibile; localStorage ramane in browser si este citit explicit de JavaScript.

Ce face SameSite=Lax?

Raspuns: Reduce trimiterea cookie-ului in anumite cereri pornite de alte site-uri, oferind o protectie de baza.

Poti sterge orice cookie cu deleteAllCookies?

Raspuns: Numai cookie-urile accesibile JavaScript-ului pentru domeniul si calea curenta; nu pe cele HttpOnly.

7.B3 Filtrele persistente (0,40 p)

Ce trebuie sa deschizi: Browser: <http://localhost:8080/produse> | VS Code: [views/pagini/produse.ejs](#); [resurse/js/produse.js](#)

Ce cere exact: Checkboxul Salveaza filtrele memoreaza valorile tuturor filtrelor in localStorage; la reintrare valorile sunt restaurate si filtrarea este reaplicata. Resetarea sterge si persistenta.

Demonstratia, in ordinea corecta

Bifeaza Salveaza filtrele.

Seteaza nume/scor/categorie si alte doua controale.

Reincarca pagina si arata ca inputurile si rezultatele revin.

Apasa Resetare, confirma, reincarca si arata ca starea nu mai revine.

Cum functioneaza in proiect

obțineStareFiltre creeaza un obiect serializabil. Valorile multiple devin vectori.

JSON.stringify transforma obiectul in text deoarece localStorage stocheaza numai siruri.

La incarcare JSON.parse reconstruieste obiectul si fiecare control este completat dupa tipul sau.

Nu se salveaza HTML-ul rezultat. Dupa restaurare se reaplica acelasi algoritm de filtrare.

try/catch elimina starea daca JSON-ul local este corupt.

Algoritmul pe care trebuie sa il intelegi

Colecteaza cele opt valori.

Serializeaza obiectul si il salveaza sub o cheie ce include URL-ul.

La incarcare citeste si parseaza cheia.

Completeaza text, range, radio, checkboxuri si selecturi.

Bifeaza optiunea de salvare si ruleaza aplicaFiltrare.

Resetarea debifeaza si removeItem sterge cheia.

Bucata-cheie: serializarea si restaurarea tuturor tipurilor de filtre:

```

77 | function obtineStareFiltre() {
78 |   return {
79 |     nume: inpNume.value,
80 |     scor: inpScor.value,
81 |     culoare: inpCuloare.value,
82 |     stoc: document.querySelector('input[name="stoc"]:checked').value,
83 |     categorii: valoriSelectate(".filtru-categorie"),
84 |     descriere: inpDescriere.value,
85 |     subcategorie: selectSubcategorie.value,
86 |     conectivitate: optiuniMultiple(selectConectivitate),
87 |   };
88 | }
89 |
90 | function salveazaStareFiltre() {
91 |   if (salveazaFiltre.checked) {
92 |     localStorage.setItem(cheieFiltrePersistente, JSON.stringify(obtineStareFiltre()));
93 |   }
94 | }
95 |
96 | function restaureazaStareFiltre() {
97 |   const stareSalvata = localStorage.getItem(cheieFiltrePersistente);
98 |   if (!stareSalvata) return;
99 |   try {
100 |     const stare = JSON.parse(stareSalvata);
101 |     inpNume.value = stare.nume || "";
102 |     inpScor.value = stare.scor || inpScor.min;
103 |     inpCuloare.value = stare.culoare || "";
104 |     inpDescriere.value = stare.descriere || "";
105 |     const radioStoc = document.querySelector(`input[name="stoc"][value="${stare.stoc} ||
"toate"]`);
106 |     if (radioStoc) radioStoc.checked = true;
107 |     document.querySelectorAll(".filtru-categorie").forEach((element) => {
108 |       element.checked = (stare.categorii || []).includes(element.value);
109 |     });
110 |     selectSubcategorie.value = stare.subcategorie || "";
111 |     Array.from(selectConectivitate.options).forEach((optiune) => {
112 |       optiune.selected = (stare.conectivitate || []).includes(optiune.value);
113 |     });
114 |     salveazaFiltre.checked = true;
115 |   } catch (eroare) {
116 |     localStorage.removeItem(cheieFiltrePersistente);
117 |   }
118 | }
119 |
120 | // Etapa 6 - cerintele 6 si 7: toate cele opt inputuri participa la filtrare.

```

Intrebari probabile si raspunsuri

De ce nu salvezi cardurile filtrate?

Raspuns: Rezultatul poate deveni inechit. Salvez intentia utilizatorului si recalculez rezultatul din datele curente.

Ce se intampla daca textul din localStorage nu este JSON valid?

Raspuns: catch sterge cheia defecta si pagina continua cu valorile implicite.

Cand se sterg filtrele?

Raspuns: Numai cand utilizatorul debifeaza salvarea sau confirma Resetarea.

Repetitia finala - traseul de 20 de minute

Minutele 0-4: Etapa 5 - galerie statica, compilare SCSS si Bootstrap customizat.

Minutele 4-7: galeria animata, backupul si validarea galeriei.

Minutele 7-12: Etapa 6 obligatorie - baza, ruta produsului, opt filtre, sortare, calcul si resetare.

Minutele 12-17: paginare, fixare/ascundere, diacritice, modal, imagini si marcatele calculate.

Minutele 17-20: carduri animate, banner cookie si filtre persistente.

Daca timpul este foarte scurt: Arata mai intai Etapa 5 obligatorie, Etapa 6 obligatorie, paginarea, modalul si filtrele persistente. Acestea combina punctaj bun cu demonstratie clara.

Checklist inainte de prezentare

Docker Desktop si PostgreSQL sunt pornite.

npm start nu afiseaza eroare fatala.

/produse contine suficiente produse pentru doua pagini.

Fereastra privata este pregatita pentru banner.

Ai verificat ca galeria, imaginile produsului si orarul au fisierele necesare.

VS Code are deschise index.js, produse.js, produse.ejs, cookies.js si custom-bootstrap.scss.

Nu este vizibil .env si nu folosesti un cont important pentru demonstratii.

Dictionarul minim pe care trebuie sa il stapanesti

EJS: motor de template care combina HTML-ul cu datele serverului.

SCSS/Sass: sursa CSS extinsa cu variabile, nesting si bucle, compilata in CSS.

DOM: reprezentarea obiectuala a paginii pe care JavaScript o poate modifica.

dataset: accesul JavaScript la atributele HTML data-.*.

Promise/await: reprezentarea si asteptarea unui rezultat asincron.

callback: functie transmisa pentru a fi apelata ulterior.

localStorage: stocare persistenta in browser, netrimisa automat serverului.

sessionStorage: stocare separata pentru tab, pierduta la inchiderea lui.

cookie: valoare cu expirare si cale, trimisa in cererile corespunzatoare.

media query: regula CSS activata de caracteristicile ecranului sau preferintele utilizatorului.

query parametrizat: SQL in care valorile sunt trimise separat prin \$1, \$2.

event delegation: un listener pe parinte gestioneaza actiunile copiilor prin event.target.